

DISPLAY and POSITION

DISPLAY

display property is used to adjust the layout of an element.

allows changing the display behavior of inline and block elements

value	description
Inline	allows an element to behave like an inline-level element
block	allows an element to behave like a block-level element
Inline-block	formats the element as inline-level but also allows to set height/width like block-level element
flex	sets the element as a flex container to have a flexible layout of its child elements
grid	sets the element as a grid container to create complex layouts
none	removes the element from the document leaving no space

DISPLAY

INLINE

allows an element to behave like an inline-level element

{display: inline}

```
p {
    background-color: beige;
    border: 1px solid black;
    height: 30px;
    width: 50px;
    display: inline;
}
```

p element p element p element p element

BLOCK

allows an element to behave like a block-level element

{display : block}

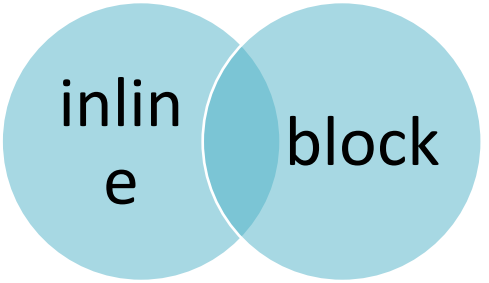
```
span {
    background-color: beige;
    border: 1px solid black;
    height: 30px;
    width: 50px;
    display: block;
}
```

span
span
span

```
li {  display: inline;
}
```

→ HTML CSS JavaScript

DISPLAY



Inline-block

formats the element as inline-level but also allows to set height/width like block-level element

```
display: inline;
```

p element p element p element p element

```
p {
    background-color: beige;
    border: 1px solid black;
    height: 50px;
    width: 100px;
    display: inline-block;
}
```

p element p element p element p element

allows to set height/width padding margin to inline element

```
span {
    background-color: beige;
    border: 1px solid black;
    height: 30px;
    width: 50px;
    padding: 50px;
    margin: 30px;
    }
```

`display: inline;`

Lorem ipsum dolor sit amet consectetur adipisicing elit. Adipisci minus corrupti ut consequuntur accusamus accusantium nesciunt labore eius nihil architecto iure dic st! Earum sint voluptate, aut in tempora quaera corporis. Facilis deserunt illum minima pari , iure! Perferendis, cum consequatur. span element Lorem ipsum dolor sit amet consectetur adipisicing elit. Reiciendis est beatae delectus vel consequatur aspernatur aliquam amet eligendi explicabo dolorem omnis iure, aliquid officiis molestias, fuga assumenda id sequi sapiente! Iste nam, expedita reprehenderit illo laudantium tempora nisi molestiae quis, id neque reiciendis, modi distinctio cumque? Quod non molestias maiores!

Effect by
- left right margin
- left right padding

`display: inline-block;`

Lorem ipsum dolor sit amet consectetur adipisicing elit. Adipisci minus corrupti ut consequuntur accusamus accusantium nesciunt labore eius nihil architecto iure dicta dolorum, nostrum magni est! Earum sint voluptate, aut in tempora quaerat error cumque? Culpa, quia corporis. Facilis deserunt illum minima pariatur earum quaerat? Aperiam, iure! Perferendis, cum

consequatur. span element Lorem ipsum dolor sit amet

consectetur adipisicing elit. Reiciendis est beatae delectus vel consequatur aspernatur aliquam amet eligendi explicabo dolorem omnis iure, aliquid officiis molestias, fuga assumenda id sequi sapiente! Iste nam, expedita reprehenderit illo laudantium tempora nisi molestiae quis, id neque

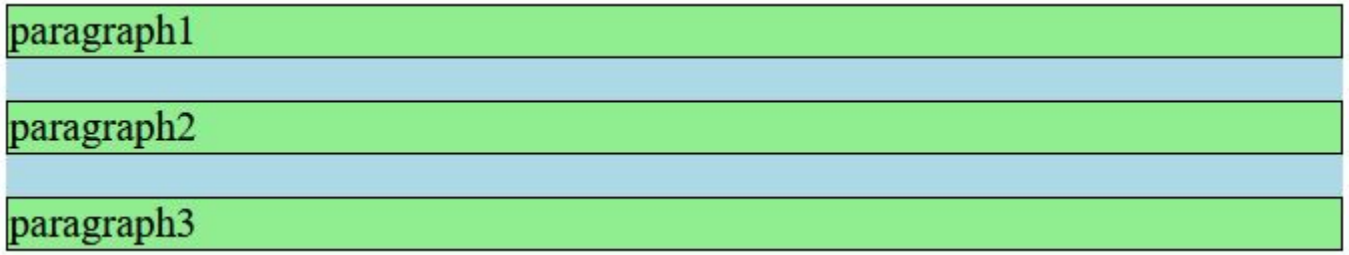
Effect by
- width / height
- left right top bottom margin
- left right top bottom padding

DISPLAY

Flexbox is a layout method for arranging items in rows or columns

Flex property allows for efficient alignment and distribution of the space between the child elements within the parent element

```
<div>
  <p>paragraph 1</p>
  <p>paragraph 2</p>
  <p>paragraph 3</p>
</div>
```




```
div {
  display: flex;
}
```





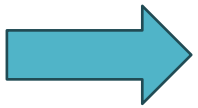
DISPLAY

none - hide an element and **remove** it from the normal flow of the document

commonly used with JavaScript to hide and show elements without deleting and recreating them

```
span{
   display:none;
}
```

Lorem ipsum dolor sit amet consectetur adipisicing elit. Deserunt temporibus odio distinctio officiis cum magnam. span element Lorem ipsum dolor sit amet, consectetur adipisicing elit. Quibusdam accusantium nemo quaerat expedita! Aperiam, neque.



Lorem ipsum dolor sit amet consectetur adipisicing elit. Deserunt temporibus odio distinctio officiis cum magnam. Lorem ipsum dolor sit amet, consectetur adipisicing elit. Quibusdam accusantium nemo quaerat expedita! Aperiam, neque.

space is removed

POSITION

position property is used to define the position of an element on a webpage

position property set with the four properties: top, left, right, and bottom



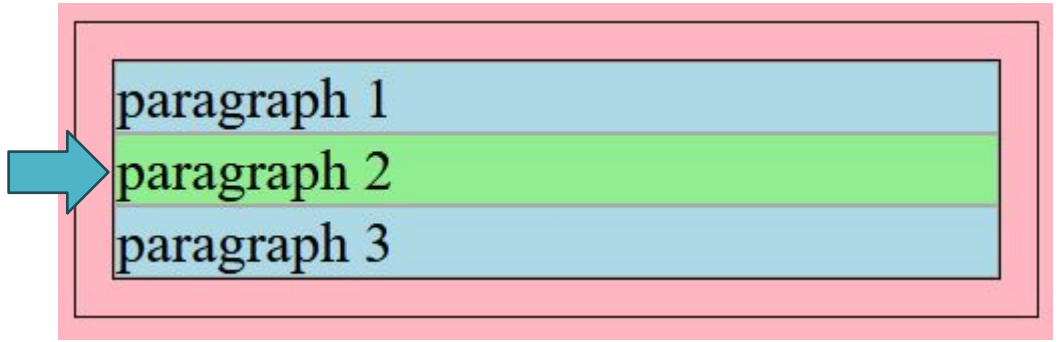
POSITION

```
<body>
    <div class="parent">
        <p>paragraph 1</p>
        <p
class="child">paragraph 2</p>
        <p>paragraph 3</p>
    </div>
</body>
```

STATIC

positioned accordingly to the normal flow of the document

```
p.child {
  position: static;
  top: 20px; /* doesn't work */
  left: 20px; /* doesn't work */
}
```

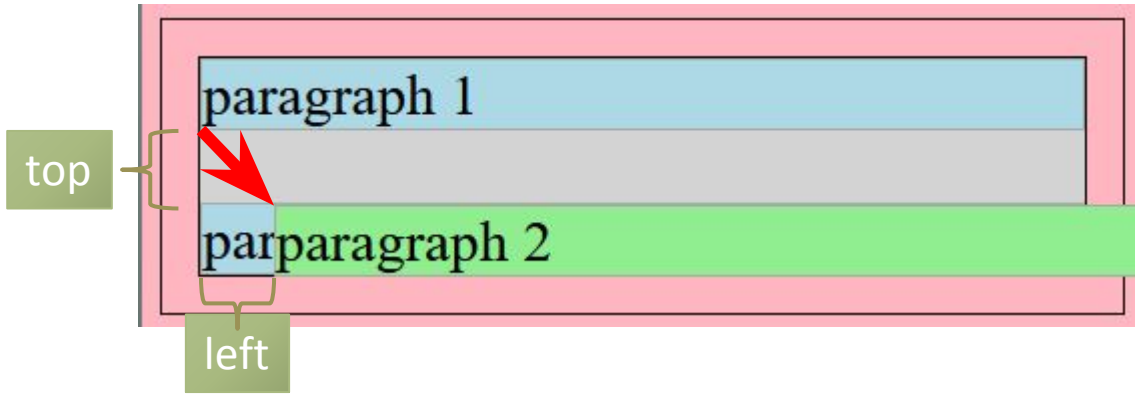


not affected by the top, bottom, left, and right properties

RELATIVE

positions the element relative to the original position

```
p.child {
  position: relative;
  top: 20px; /* from original position */
  left: 20px; /* from original position */
}
```

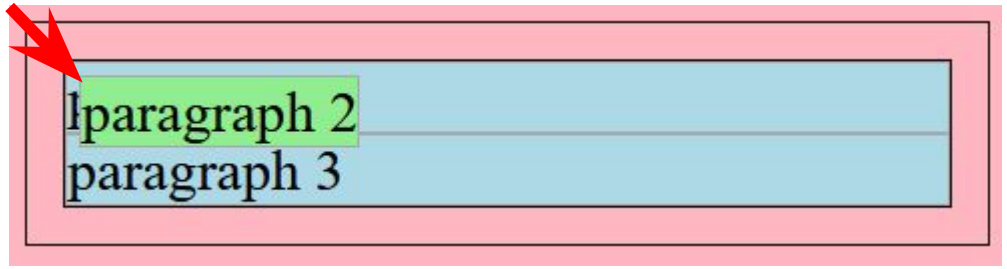


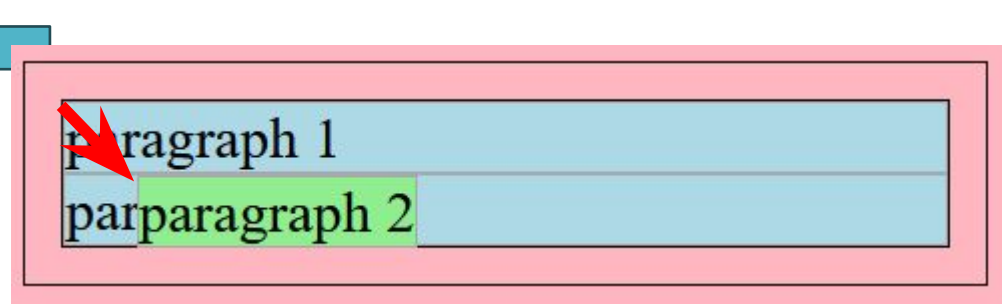
POSITION

```
p.child {
    position: absolute;
    top: 20px; /* from positioned parent */
    left: 20px; /* from positioned parent */
}
```

absolute

removes the element completely from its normal flow in the document and positioned relative to their closest positioned **parent** element





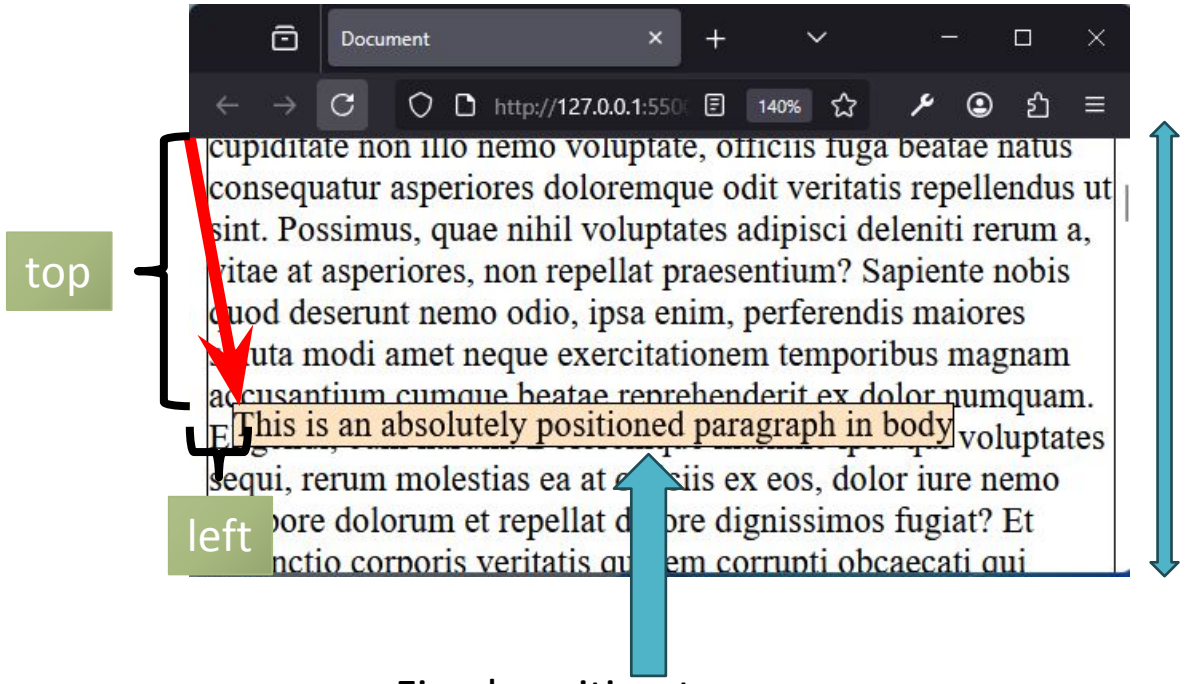
the element with absolute, needs a parent somewhere to be positioned relative

```
div {
    position: relative;
}
```

If no positioned parent the element position to viewport

POSITION

fixed value positions an element to remain fixed in the same position, even when the page is scrolled

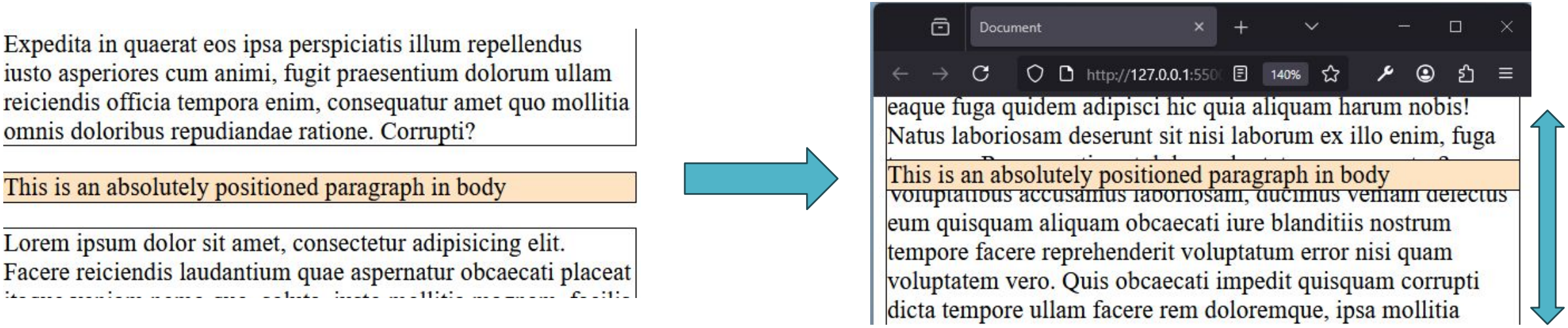


POSITION

sticky :

behaves like relative positioning until the element reaches a certain scroll point.

After that, the element sticks to the top/bottom of the viewport like a fixed element.

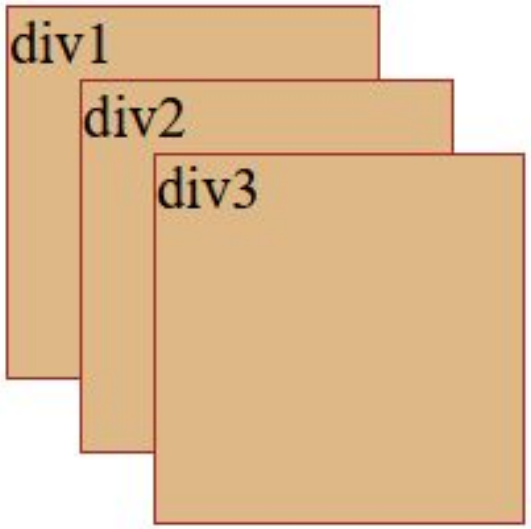


Z-index

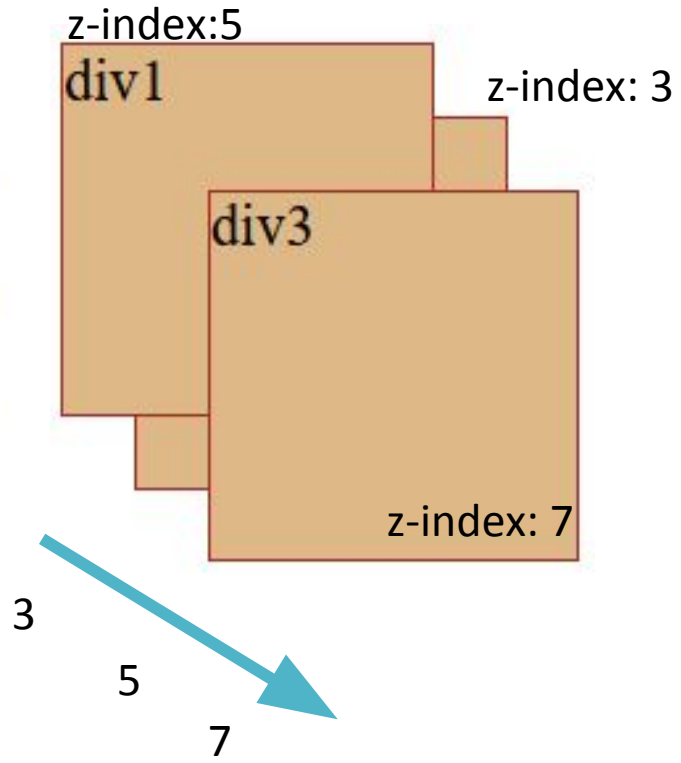
z-index property is used to control the stacking order of the positioned elements

Z-index value	description
auto	determines the stacking order based on the element's position in the HTML document (default value)
number	sets the stacking order of an element, negative values are allowed
initial	sets the property value to the default
inherit	inherits the property value from the parent

```css
div.div1 {
    position: absolute;
    top: 10px;
    left: 10px;
}
div.div2 {
    position: absolute;
    top: 30px;
    left: 30px;
}
div.div3 {
    position: absolute;
    top: 50px;
    left: 50px;
}
```




```css
div.div1{
    z-index: 5;
}
div.div2{
    z-index: 3;
}
div.div3{
    z-index: 7;
}
```

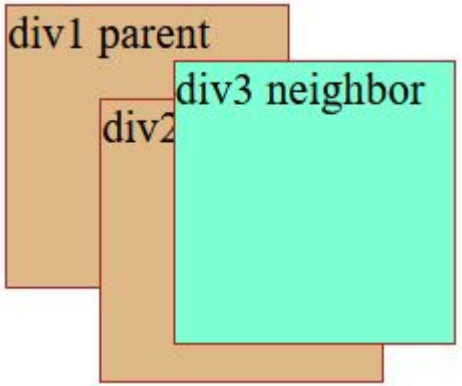


If element B comes after element A then, element B will be stacked on top of element A .
Elements with the same z-index value are stacked based on their order in the HTML document.
The negative values are allowed

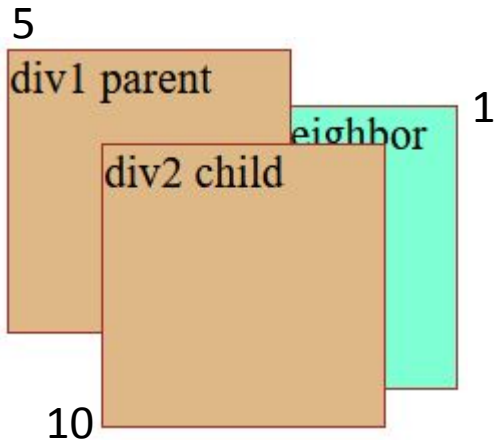
```css
div.div1 {
    position: absolute;
    top: 10px;
    left: 10px;
}

div.div2 {
    position: absolute;
    top: 30px;
    left: 30px;
}

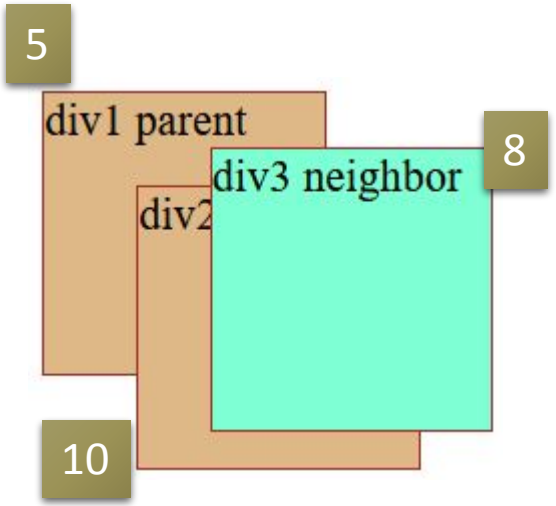
div.div3 {
    position: absolute;
    top: 30px;
    left: 70px;
}
```



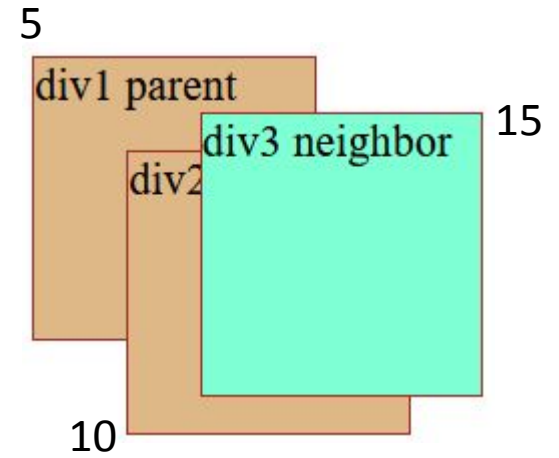

```html
<div class="div1">div1 parent
    <div class="div2">div2 child</div>
</div>
<div class="div3">div3 neighbor</div>
```




```css
div.div1 {z-index: 5;}
div.div2 {z-index: 10;}
div.div3 {z-index: 1;}
```




```css
div.div1 {z-index: 5;}
div.div2 {z-index: 10;}
div.div3 {z-index: 8;}
```




```css
div.div1 {z-index: 5;}
div.div2 {z-index: 10;}
div.div3 {z-index: 15;}
```

The child element cannot stack above an element that has a higher z-index value than its parent element

OVERFLOW

overflow property is used to adjust the content when its **size is too big** relative to the element box

values	
visible	allows the content to overflow (default value)
hidden	hides the overflowing content
scroll	adds scrollbars to the element box and allows to scroll the content
auto	adds scrollbars only when necessary
initial	sets the property value to the default value
inherit	inherits the property value from the parent element

Lorem ipsum dolor sit amet consectetur, adipisicing elit. Voluptatem laboriosam earum maxime enim eaque, eligendi aspernatur cumque mollitia id dolores architecto eveniet dolorem iste ex fugit excepturi. Molestiae expedita iste dolores architecto non illo laborum sapiente, eveniet doloribus quam dicta dolorem inventore eaque, harum amet. Quae perferendis tenetur veniam vitae voluptatibus expedita ipsum consectetur animi sunt eveniet deserunt voluptates ullam consequuntur, quasi modi. In expedita rem rerum omnis pariatur voluptatem. Illum ad maxime nisi expedita a iure cum, quae, aut molestiae placeat et officiis reprehenderit. Veritatis quibusdam reiciendis quo exercitationem magni ea consequatur omnis, repudiandae sequi deserunt, praesentium corrupti vel.

visible

Lorem ipsum dolor sit amet consectetur, adipisicing elit. Voluptatem laboriosam earum maxime enim eaque, eligendi aspernatur cumque mollitia id dolores architecto eveniet dolorem iste ex fugit excepturi. Molestiae expedita iste dolores

hidden

Lorem ipsum dolor sit amet consectetur, adipisicing elit. Voluptatem laboriosam earum maxime enim

scroll

Lorem ipsum dolor sit, amet consectetur adipisicing elit. Perspiciatis dignissimos,

Lorem ipsum dolor sit, amet consectetur.

auto

Lorem ipsum dolor sit amet consectetur, adipisicing elit. Voluptatem laboriosam earum maxime enim

Lorem ipsum dolor sit amet consectetur,

overflow-x adjust the content overflow in only the horizontal direction

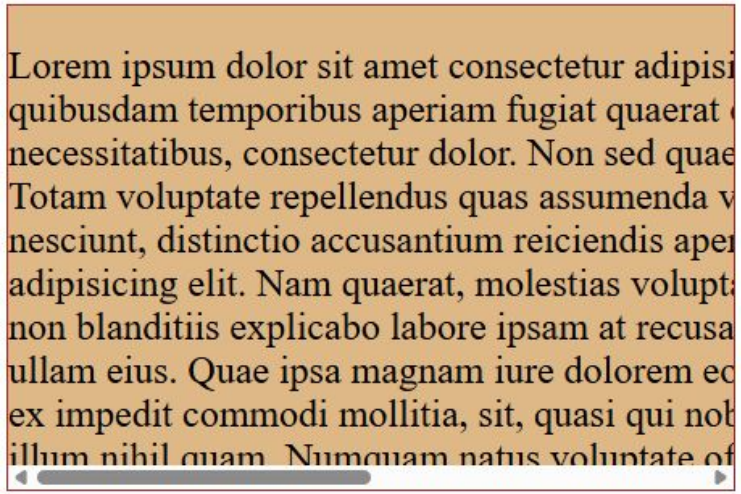
overflow-y adjust the content overflow in only the vertical direction

```
div {
    width: 300px;
    height: 200px;
    overflow-x: scroll;
    overflow-y: hidden;
}

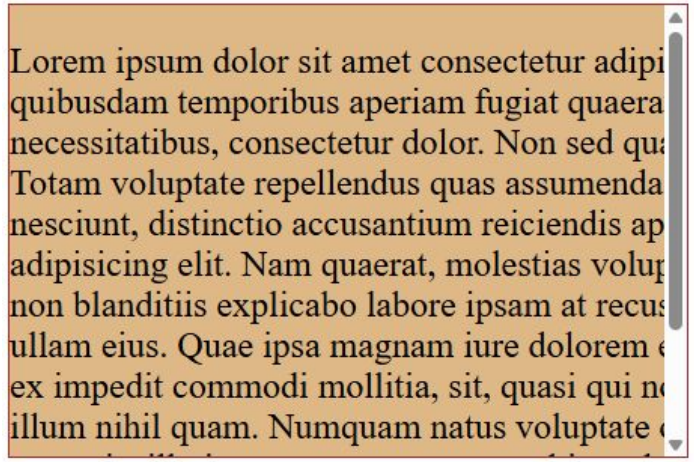
p {
    width: 600px;
}
```

Define in

```
<div>
    <p>Lorem ipsum …</p>
</div>
```




```
overflow-x: scroll;
overflow-y: hidden;
```




```
overflow-x: hidden;
overflow-y: scroll;
```



FLOAT LAYOUT

FLOAT

float property is used to specify how an element floats within a parent element.

Value	
none	doesn't float (default value)
left	floats on the left side of its containing block
right	floats on the right side of its containing block
initial	the value is set to the default value
inherit	inherits floating property from its parent element

float:none;

Lorem ipsum, dolor sit amet consectetur adipisicing elit. Dolorem maxime harum tempore in? Nobis perferendis maiores quaerat



amet doloremque ex. Lorem ipsum dolor sit amet consectetur adipisicing elit. Exercitationem eaque quidem, maiores voluptatum saepe tempora dolore officiis error libero necessitatibus.

Lorem ipsum dolor sit amet  Lorem ipsum dolor sit amet consectetur adipisicing elit. Exercitationem eaque quidem, maiores voluptatum saepe tempora dolore officiis error libero necessitatibus.

float:left;

Lorem ipsum, dolor sit amet consectetur adipisicing elit. Dolorem maxime harum tempore in? Nobis perferendis maiores quaerat



amet doloremque ex. Lorem ipsum dolor sit amet consectetur adipisicing elit. Exercitationem eaque quidem, maiores voluptatum saepe tempora dolore officiis error libero necessitatibus.



Lorem ipsum dolor sit amet Lorem ipsum dolor sit amet consectetur adipisicing elit. Exercitationem eaque quidem, maiores voluptatum saepe tempora dolore officiis error libero necessitatibus.

float:right;

Lorem ipsum, dolor sit amet consectetur adipisicing elit. Dolorem maxime harum tempore in? Nobis perferendis maiores quaerat amet doloremque ex. Lorem ipsum dolor sit amet consectetur adipisicing elit. Exercitationem eaque quidem, maiores voluptatum saepe tempora dolore officiis error libero necessitatibus.



Lorem ipsum dolor sit amet Lorem ipsum dolor sit amet consectetur adipisicing elit. Exercitationem eaque quidem, maiores voluptatum saepe tempora dolore officiis error libero necessitatibus.



FLOAT- inline element

The floated inline text elements

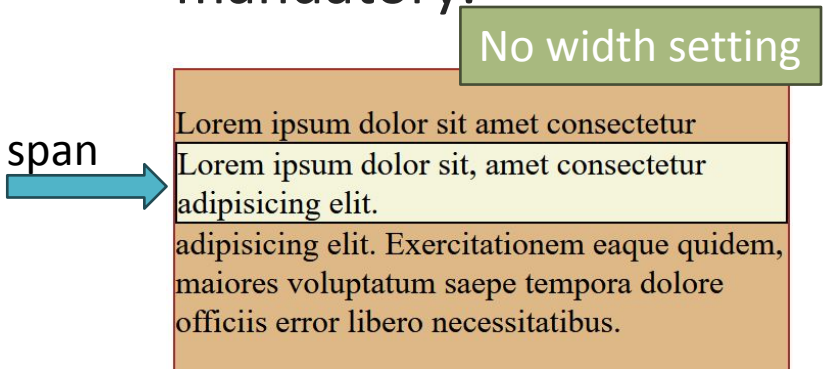
The floated inline elements behave as block elements.

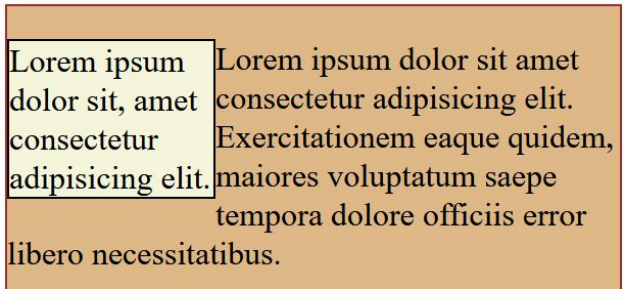
We can add width, height, margin, and padding on all sides.

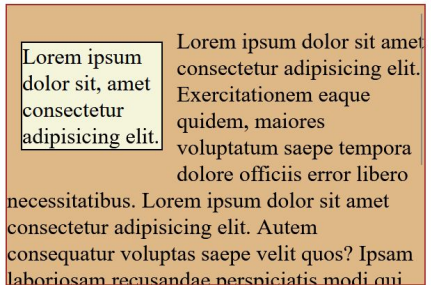
The margin in the floated element does not collapse. In the normal flow, the adjacent top and bottom margins collapse and take the value of the larger margin.

should always be assigned a **width** value. Otherwise, they will expand to fit their content

Images have their inherent width and height, so adding width and height is not mandatory.





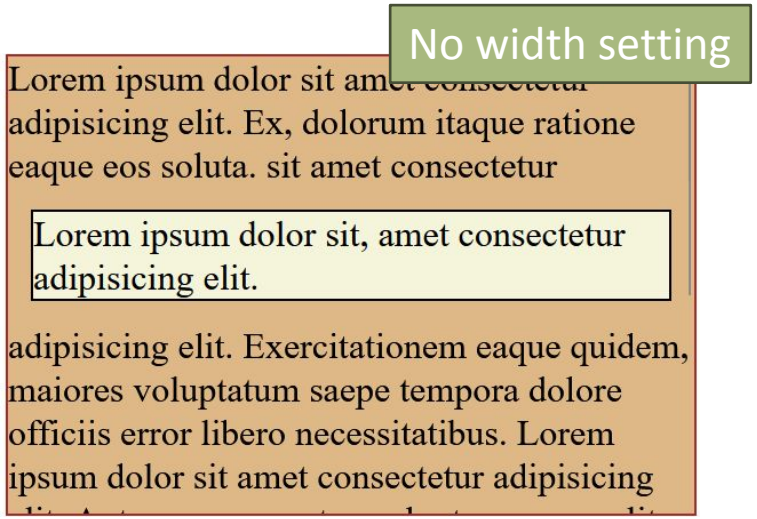


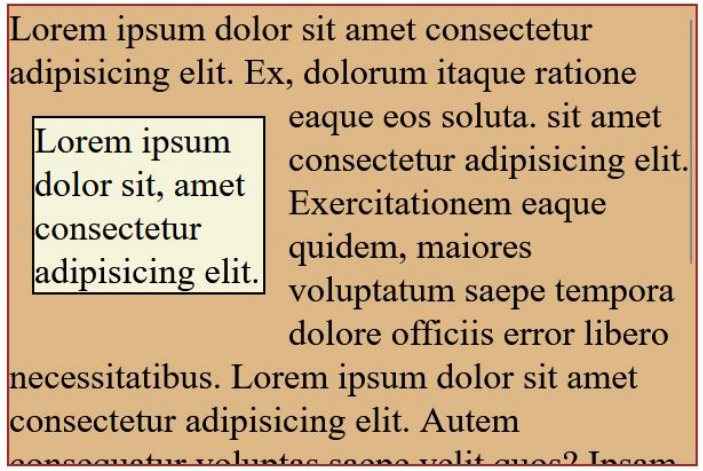
FLOAT – block element

The floated block elements

The block-level element should be assigned a **width** value when using the float property. Otherwise, the default width value is auto, which causes it to fill the entire width of the parent element.

The floating block element does not float higher than its original reference in the document.



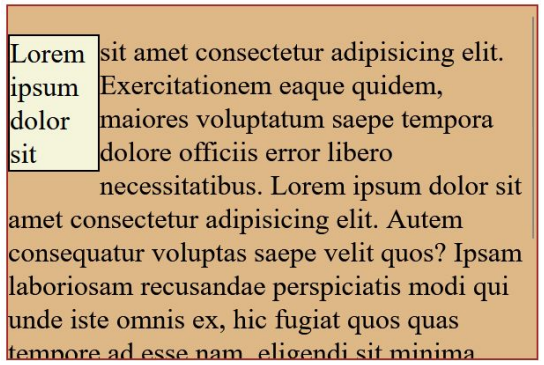


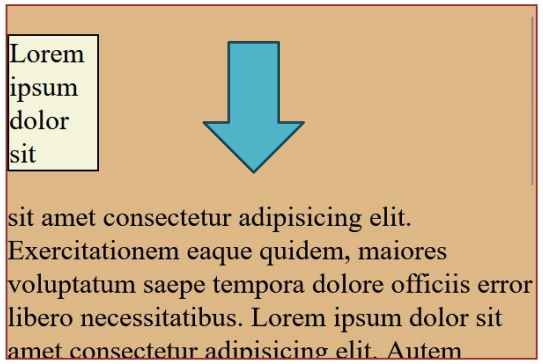
CLEAR FLOAT

clear property controls the behavior of elements that are adjacent to floated elements.

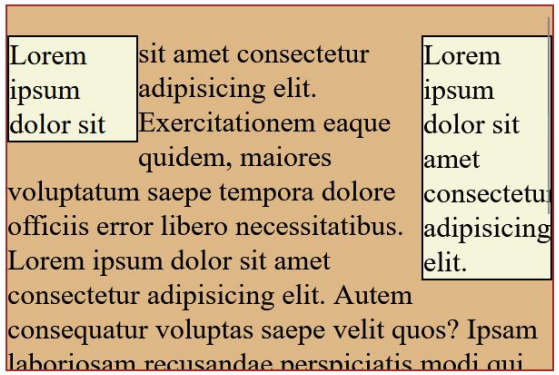
It allows to turn off the wrapping of the text and moves the element below the floating element.

```
clear: none | left | right | both | initial | inherit;
```

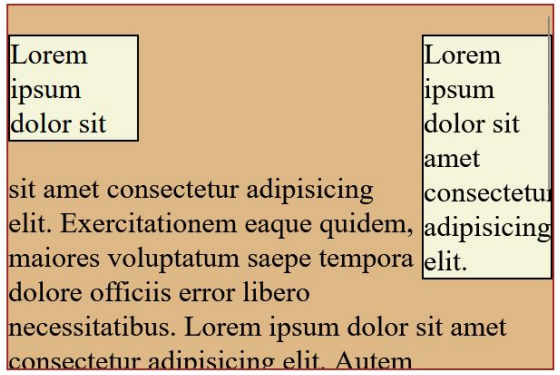




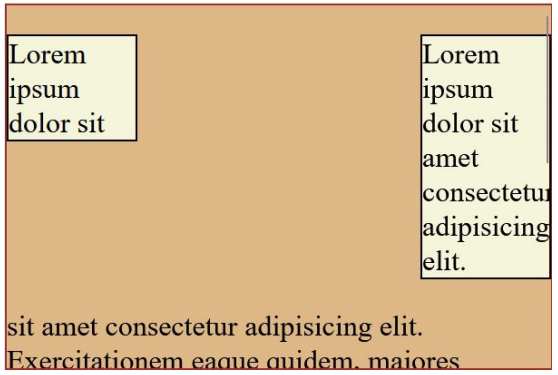
clear:none;



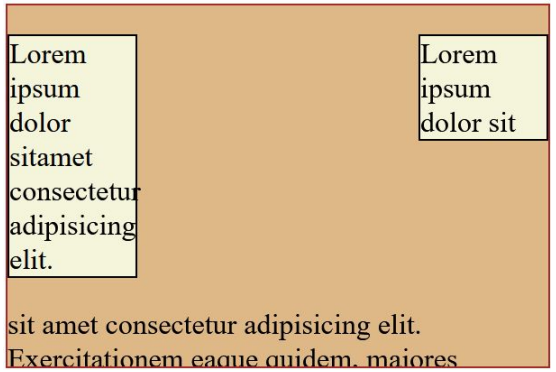
clear:left;



clear:right;



clear:both;

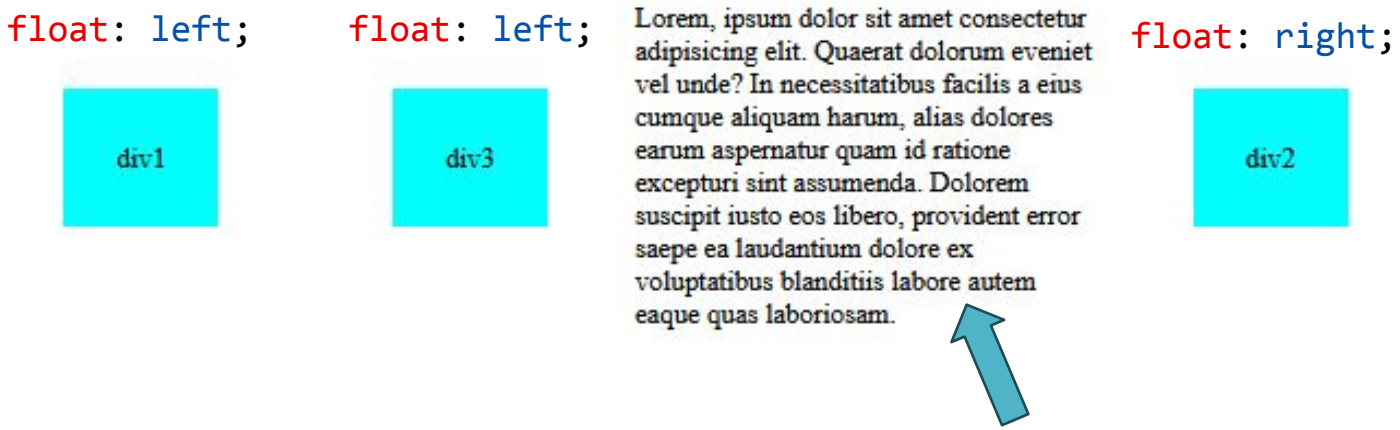


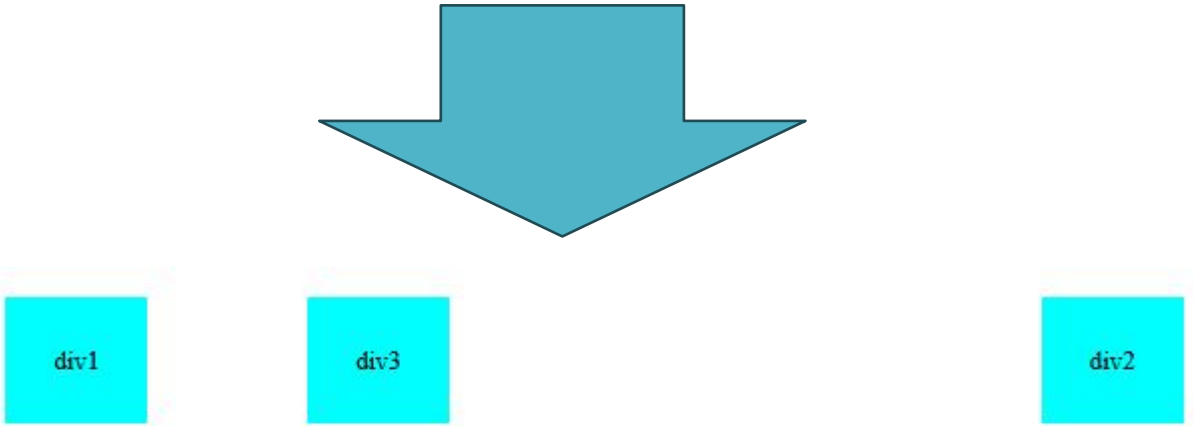
values	description
none	(default) allows the paragraph to wrap around the floated image element
left	The element is pushed below left floated elements
right	The element is pushed below right floated elements
both	The element is pushed below both left and right floated elements
initial	Sets this property to its default value
inherit	Inherits this property from its parent element

```
<div>div1</div>
<div class="right">div2</div>
<div>div3</div>
<p>Lorem,...</p>
```

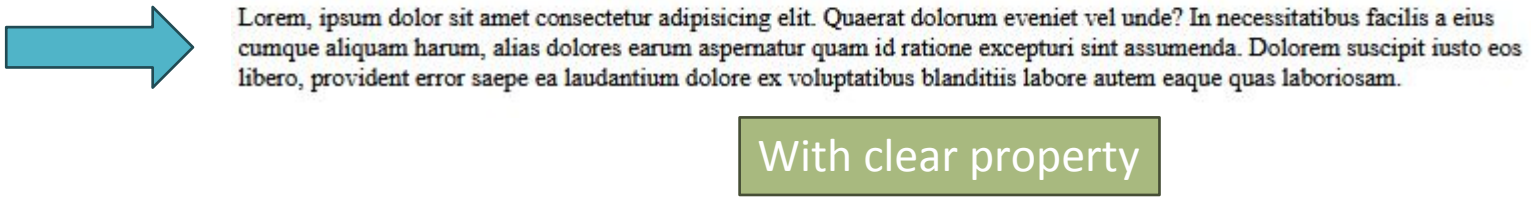
```
div {
    margin:50px;
    padding:30px;
    background-color: aqua;
    float: left;
}

.right {
    float: right
}
```






```
p {
    clear:both;
}
```

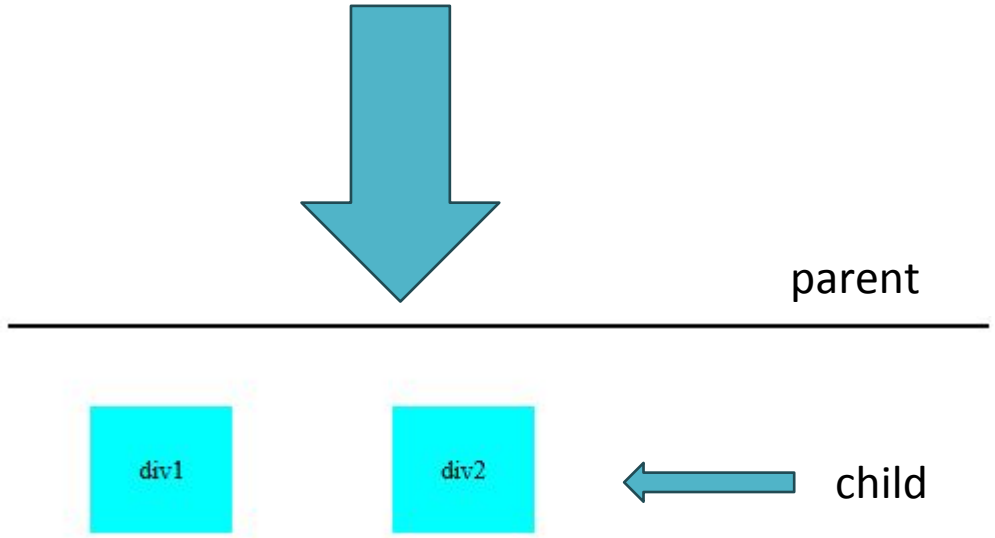


Issues with CSS float

The parent or container box collapses completely when all its contents are floated

```
.parent
{
    border: 1px solid black;
}
.left {
    margin:50px;
    padding:30px;
    background-color: aqua;
    float: left;
}
```

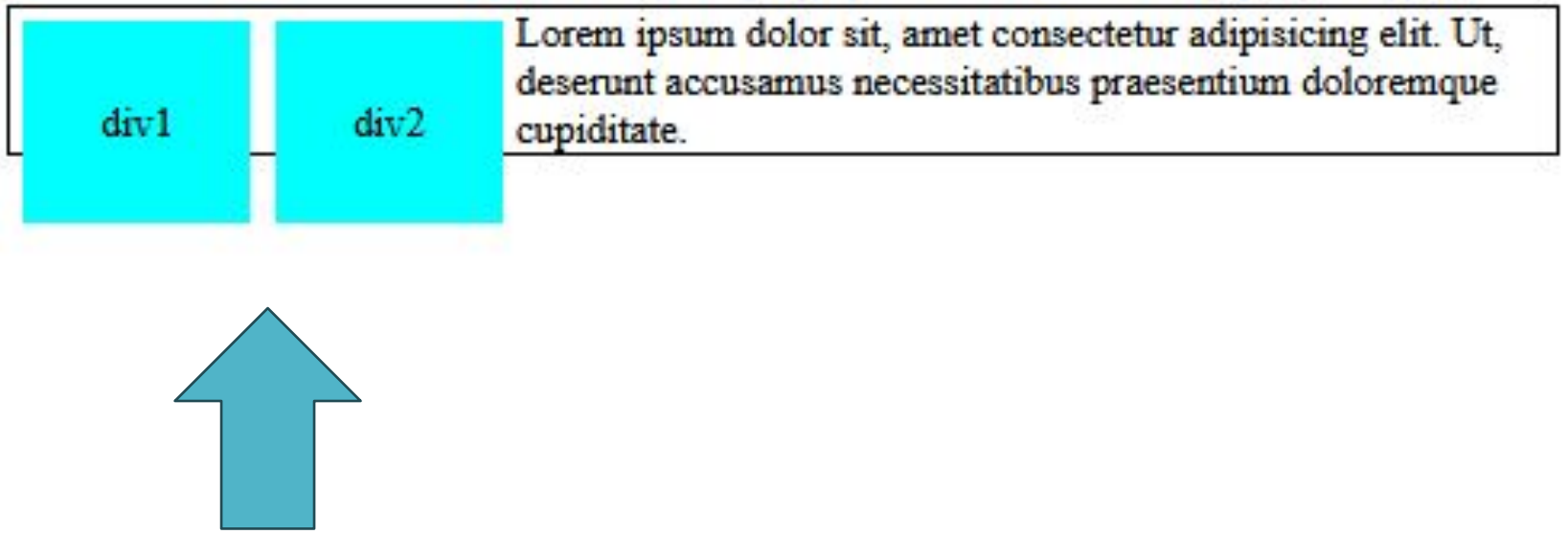
```
<div class="parent">
    <div class="left">div1</div>
    <div class="left">div2</div>
</div>
```



`float: left;` `float: left;`

Issues with CSS float

The floated element overflows the parent or container element if its height exceeds the height of the parent element

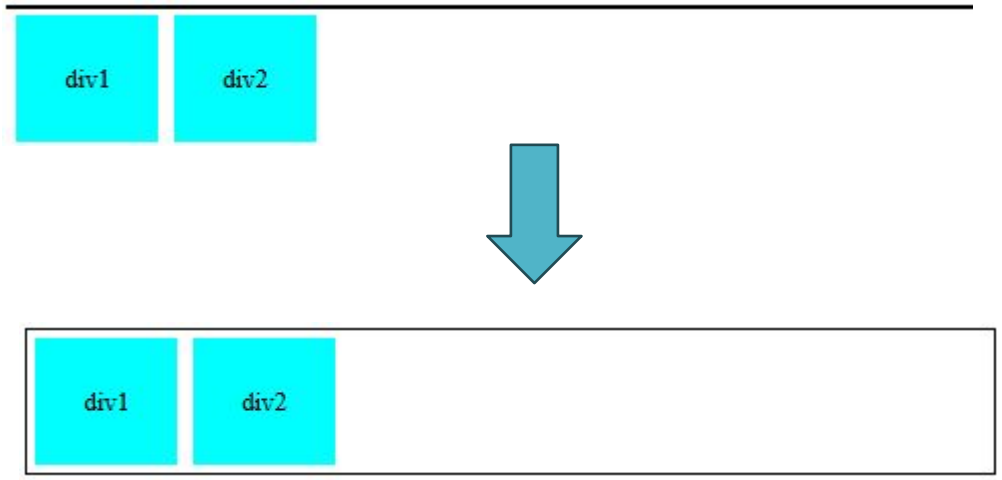


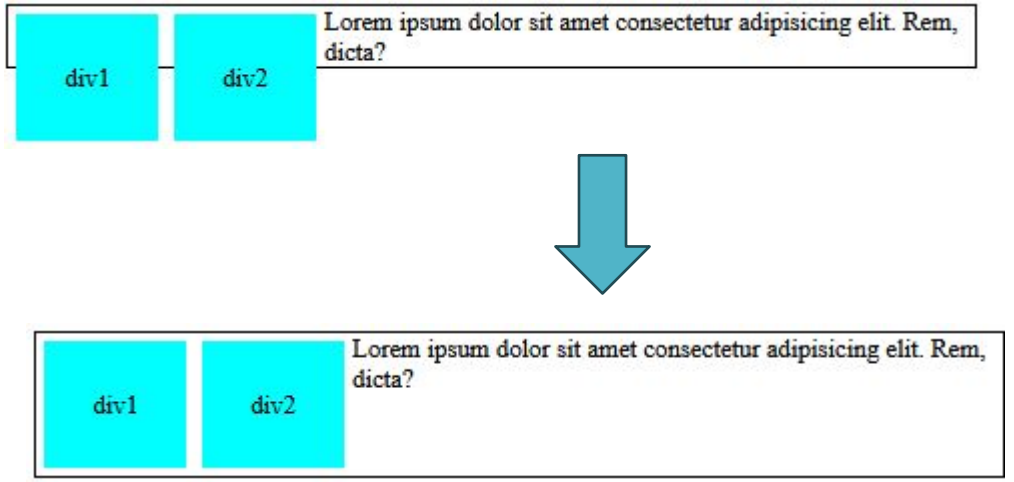
Using Empty Div Element

to solve the collapsing and overflow issues

```
.empty {
        clear: both;
}
```

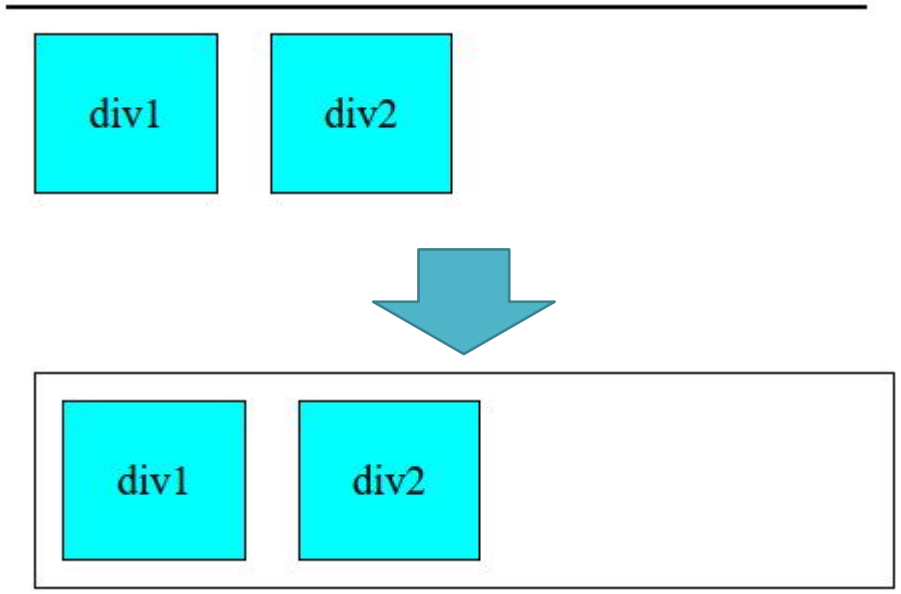
```
<div class="parent">
    <div class="right">div1</div>
    <div class="right">div2</div>
    <div class="empty"></div>
</div>
```

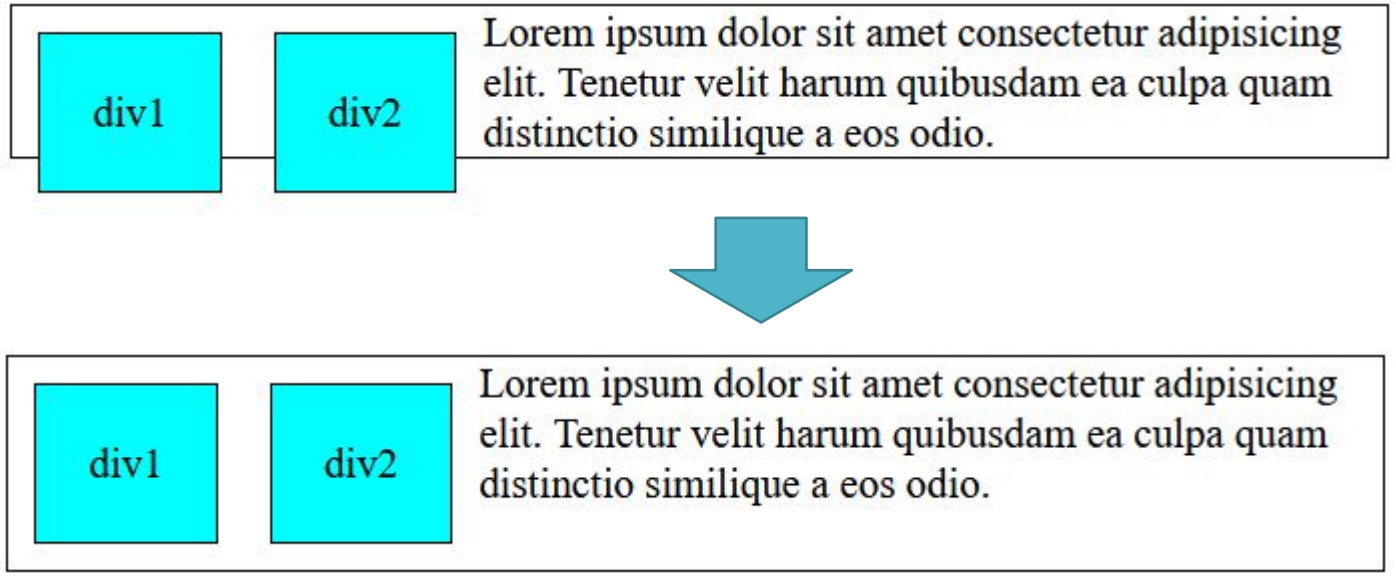




Using CSS overflow

```
.parent
    {
      overflow: auto;
      border: 1px solid black;
    }
```

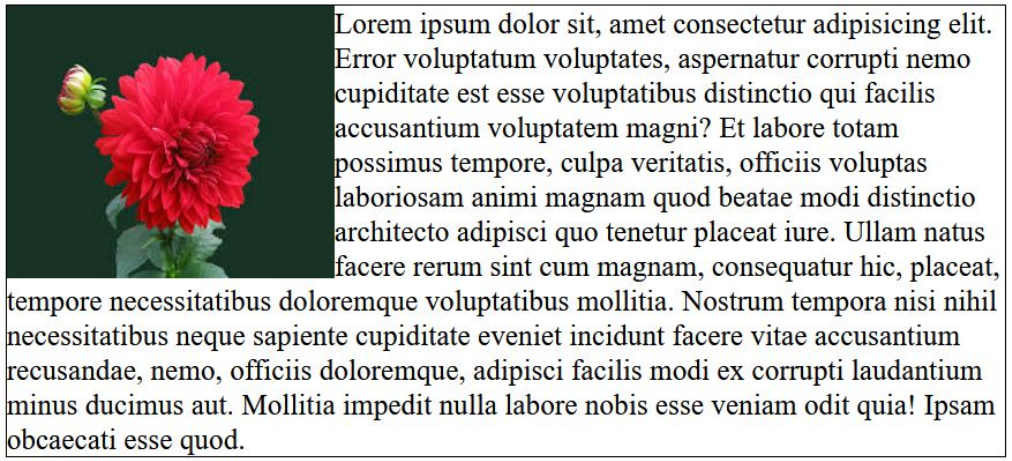




Float Layout

The float property can be used to create various layout designs,

text wrapping around an image, multi-column layouts, and the image gallery.

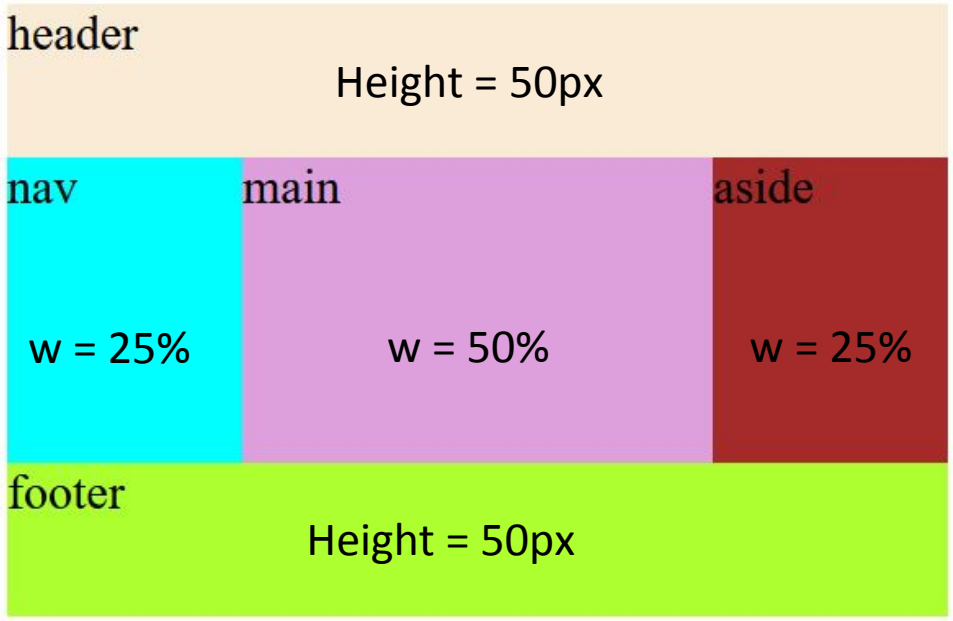


Wrapping the text around the Image



Creating an Image Gallery

Float Layout



Creating
Layouts

Semantic tag

```
<body>
   <header>header</header>
   <nav>nav</nav>
   <main>main</main>
   <aside>aside</aside>
   <footer>footer</footer>
</body>
```



FLEXBOX

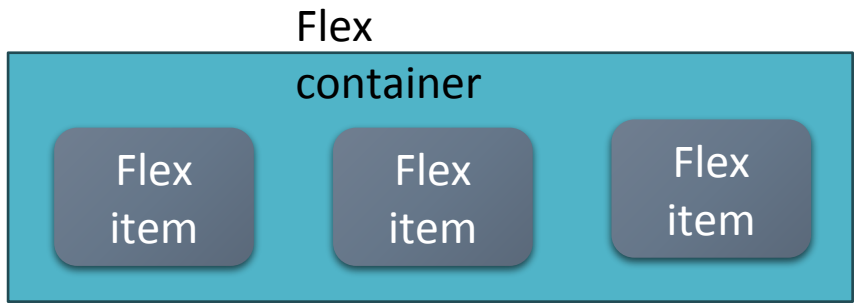
FLEXBOX

Flexbox is Flexible Box Layout module

Flexbox is a layout method for arranging items in rows or columns

A flexbox always consists of






```
<div>
   <div>1</div>
   <div>2</div>
   <div>3</div>
</div>
```





FLEX container

The flex container becomes flexible by setting the display property to flex




```
.flex-container {
  display: flex;
}

.flex-container > div {
  background-color: #f1f1f1;
  margin: 10px;
  padding: 20px;
}
```

```
<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
</div>
```

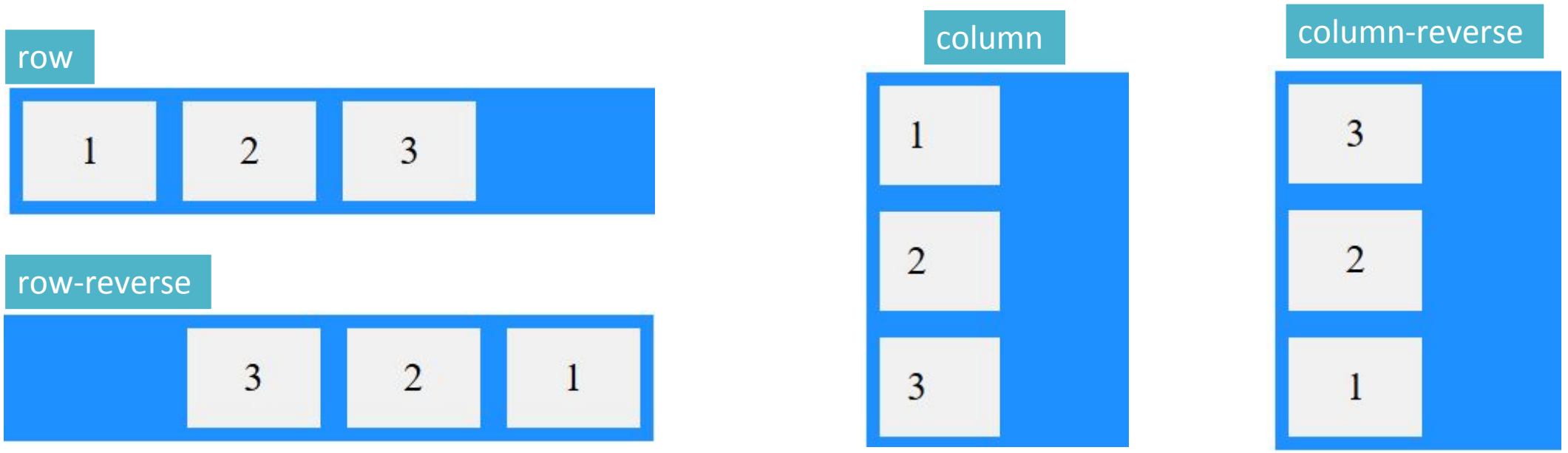
The CSS properties we use for the flex container are:

flex-direction
flex-wrap
flex-flow
justify-content
align-items
align-content

FLEX container - flex-direction

flex-direction property specifies the display-direction of flex items

flex-direction : row | column | row-reverse |column-reverse;



FLEX container - flex-wrap

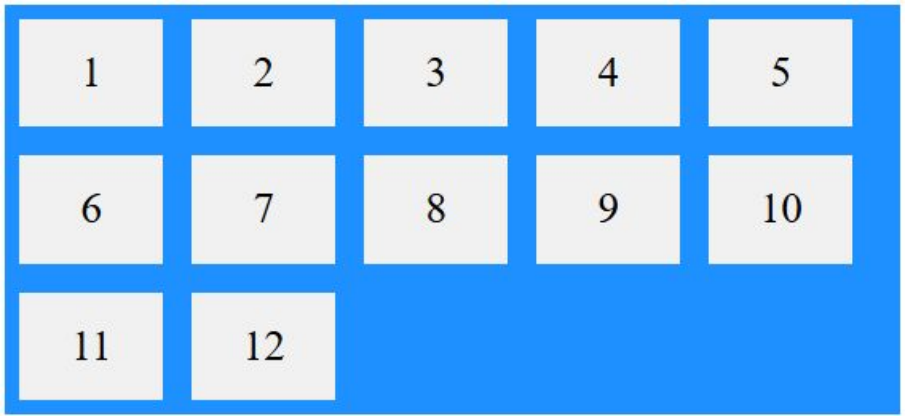
flex-wrap property specifies whether the flex items should wrap or not, if there is not enough room for them on one flex line

flex-wrap : nowrap | wrap | wrap-reverse;

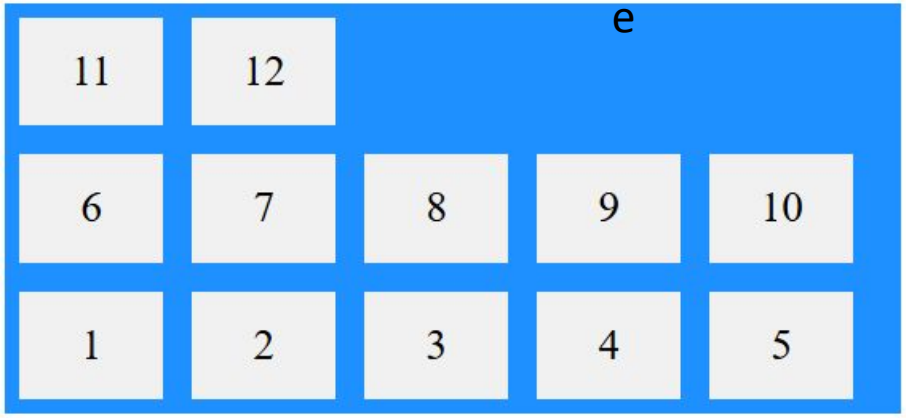
nowrap



wrap



wrap-reverse



FLEX container - flex-flow

flex-flow property is a shorthand property for setting both the flex-direction and flex-wrap properties

```
.flex-container {
  display: flex;
  flex-direction: row;
  flex-wrap:wrap;
}
```



```
.flex-container {
  display: flex;
  flex-flow: row wrap;
}
```

FLEX container - justify-content

justify-content property is used to align the flex items when they do not use all available space on the main-axis (horizontally)

```
justify-content:center|flex-start|flex-end|space-around|space-between|space-evenly;
```

center



space-around



flex-start



space-between



flex-end



space-evenly

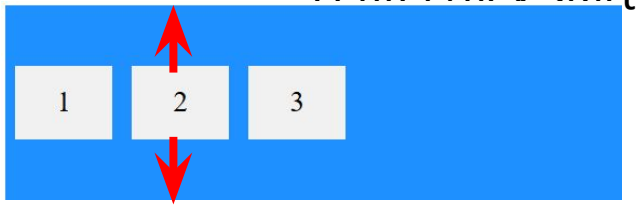


FLEX container – align-items

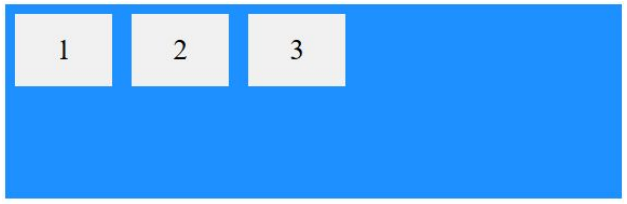
align-items property is used to align the flex items when they do not use all available space on the cross-axis (vertically)

align-items:
center | flex-start | flex-end | stretch | baseline | normal;

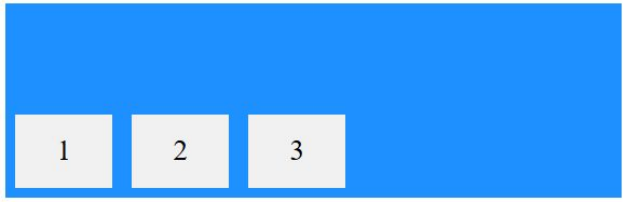
center



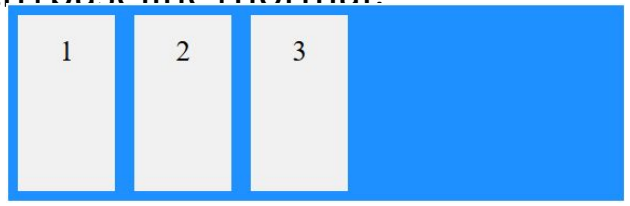
flex-start



flex-end

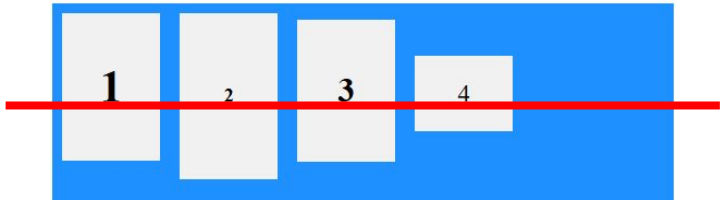


stretch



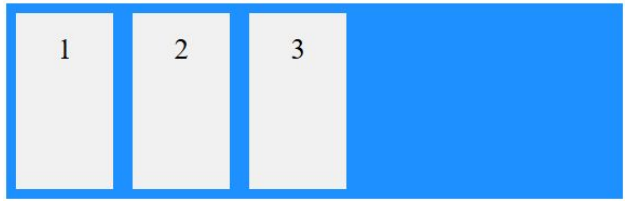
equal to "normal"

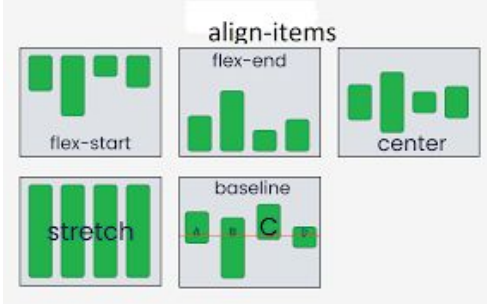
baseline



baseline

normal

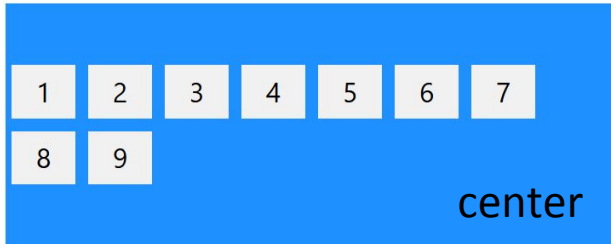


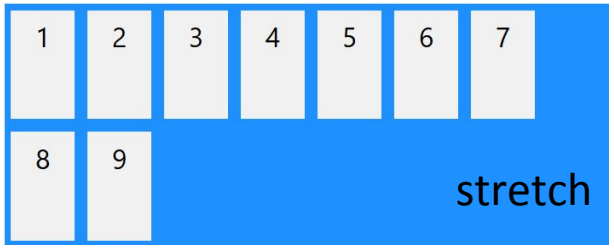


FLEX container - align-content

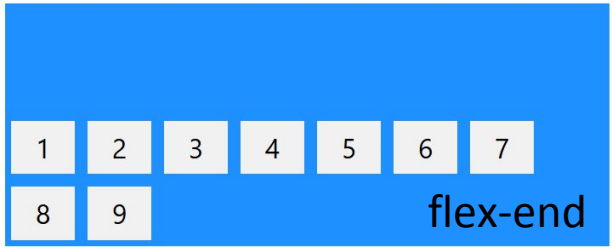
align-content property is used to align the flex lines

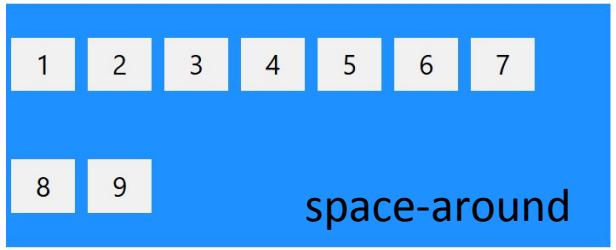
align-content :center|stretch|flex-start|flex-end|space-around|space-between|space-evenly;

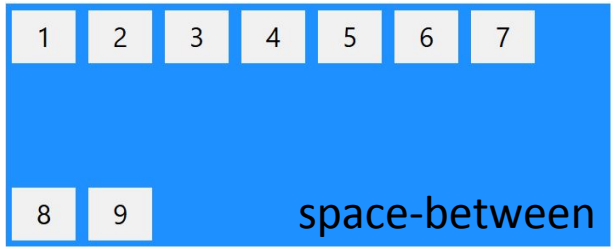


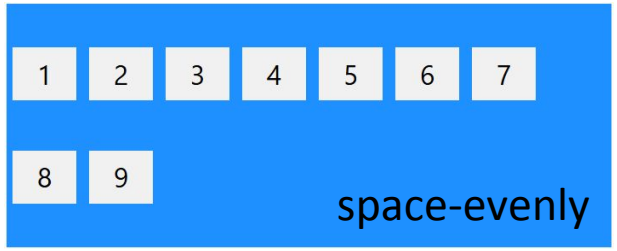








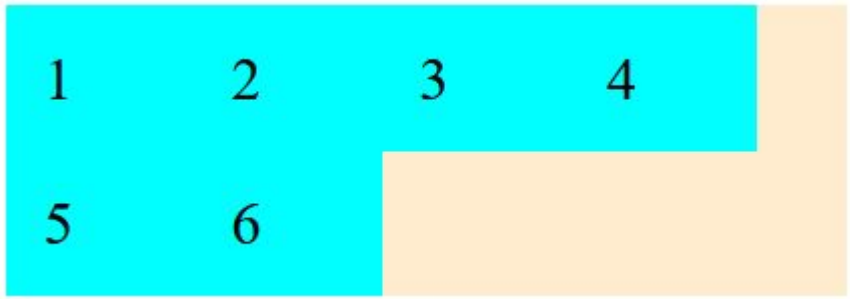




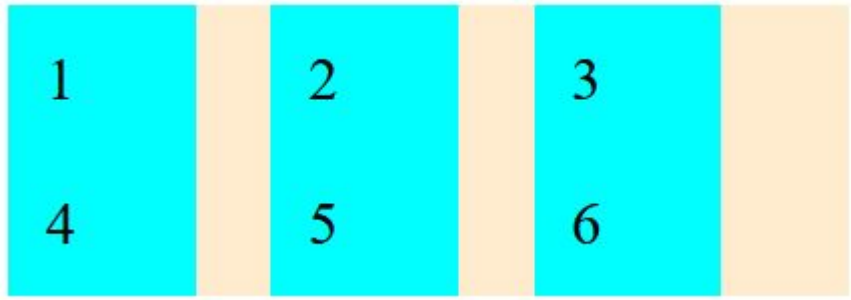
FLEX container - gap

row-gap and column-gap properties control the spacing between the flex items within the flex container.

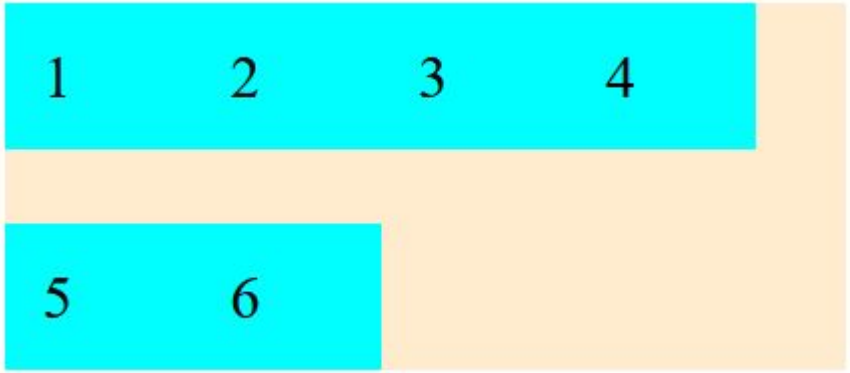
```
.container{
      display: flex;
      flex-wrap: wrap;
}
```



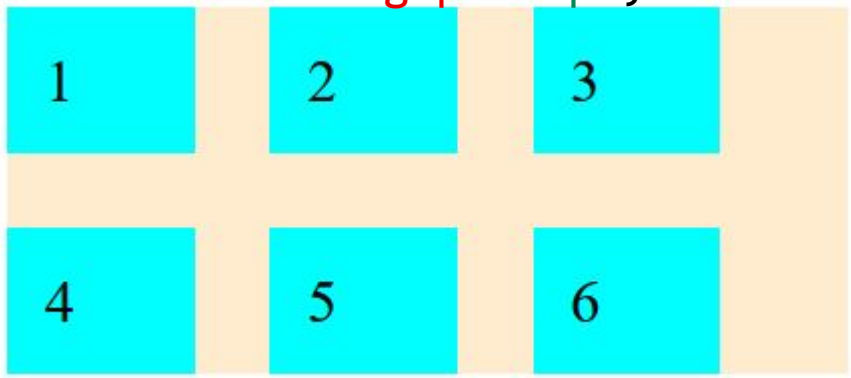
column-gap: 20px;



row-gap: 20px;



row-gap: 20px;
column-gap: 20px;

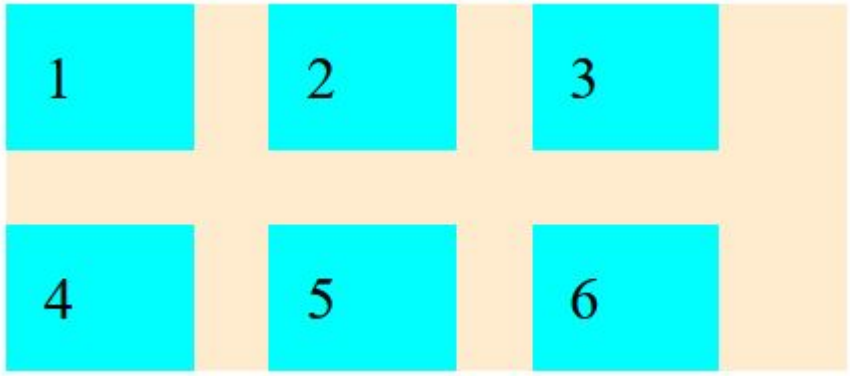


The gap is the shorthand property that allows setting both row and column gaps between the flex items

```
row-gap: 20px;
column-gap: 20px;
```



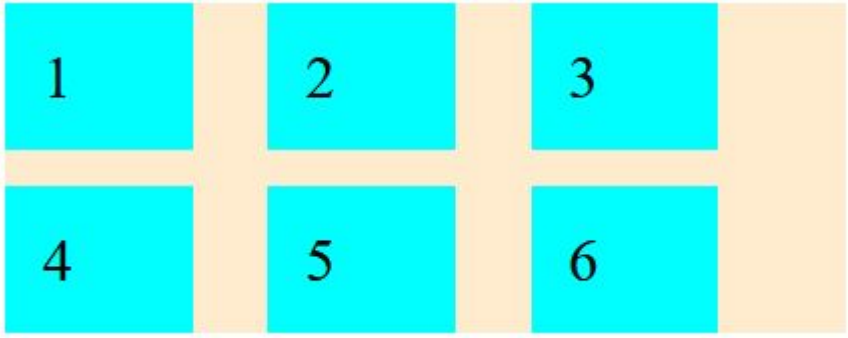
```
gap: 20px;
```




```
row-gap: 10px;
column-gap: 20px;
```



```
gap: 10px 20px;
```



FLEX item

All direct child elements of a flex container automatically becomes flex items

```
.flex-container {
  display: flex;
  background-color: #f1f1f1;
}

<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
</div>
```



flex items

properties of flex items are

- order
- flex-grow
- flex-shrink
- flex-basis
- flex
- align-self

FLEX item-order

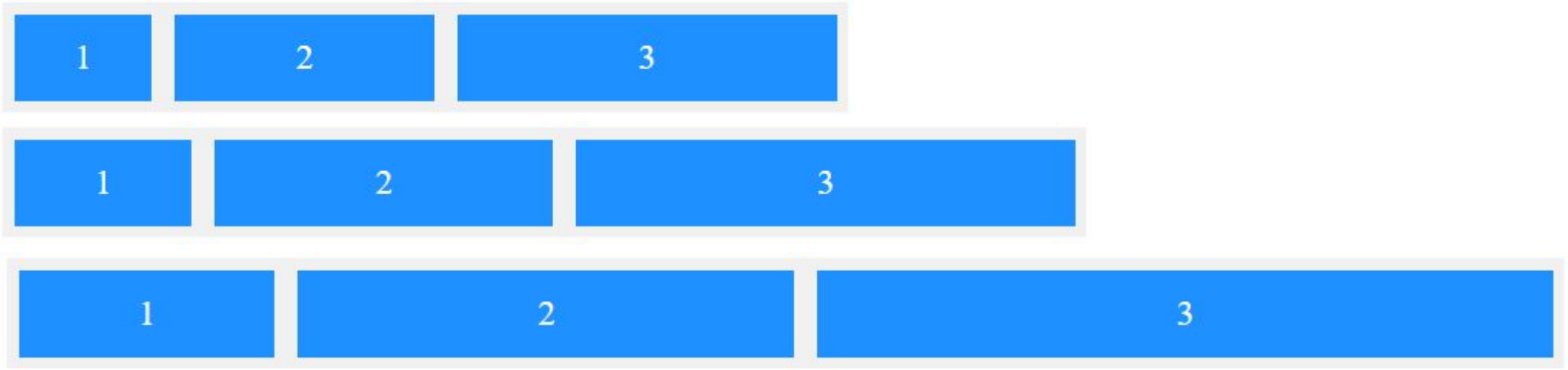
order property specifies the order of the flex items inside the flex container




```
<div class="flex-container">
  <div style="order: 3">1</div>
  <div style="order: 2">2</div>
  <div style="order: 4">3</div>
  <div style="order: 1">4</div>
</div>
```

FLEX item - flex-grow

flex-grow property specifies how much a flex item will grow relative to the rest of the flex items




```
<div class="flex-container">
  <div style="flex-grow: 1">1</div>
  <div style="flex-grow: 2">2</div>
  <div style="flex-grow: 3">3</div>
</div>
```

FLEX item – flex-shrink

flex-shrink property specifies how much a flex item will shrink relative to the rest of the flex items

default value is 1

3



Resize smaller 3 times relative to the others

FLEX item – flex-basis

flex-basis property specifies the initial length of a flex item




```
<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div style="flex-basis: 200px">3</div>
  <div>4</div>
</div>
```

FLEX item - flex

flex property is a shorthand property for the flex-grow, flex-shrink, and flex-basis properties

```
<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div style="flex: 0 0 200px">3</div>
  <div>4</div>
</div>
```

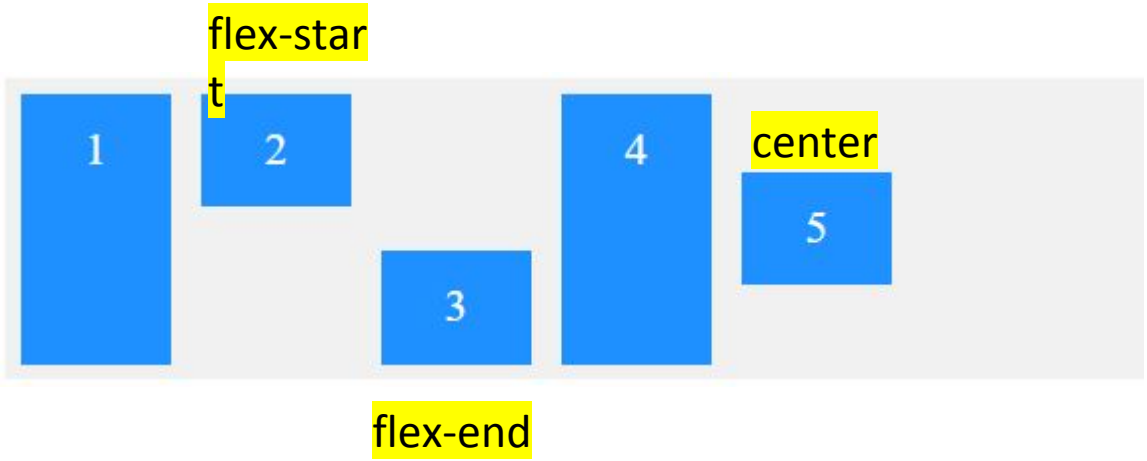


flex-grow flex-shrink flex-basis

FLEX item – align self

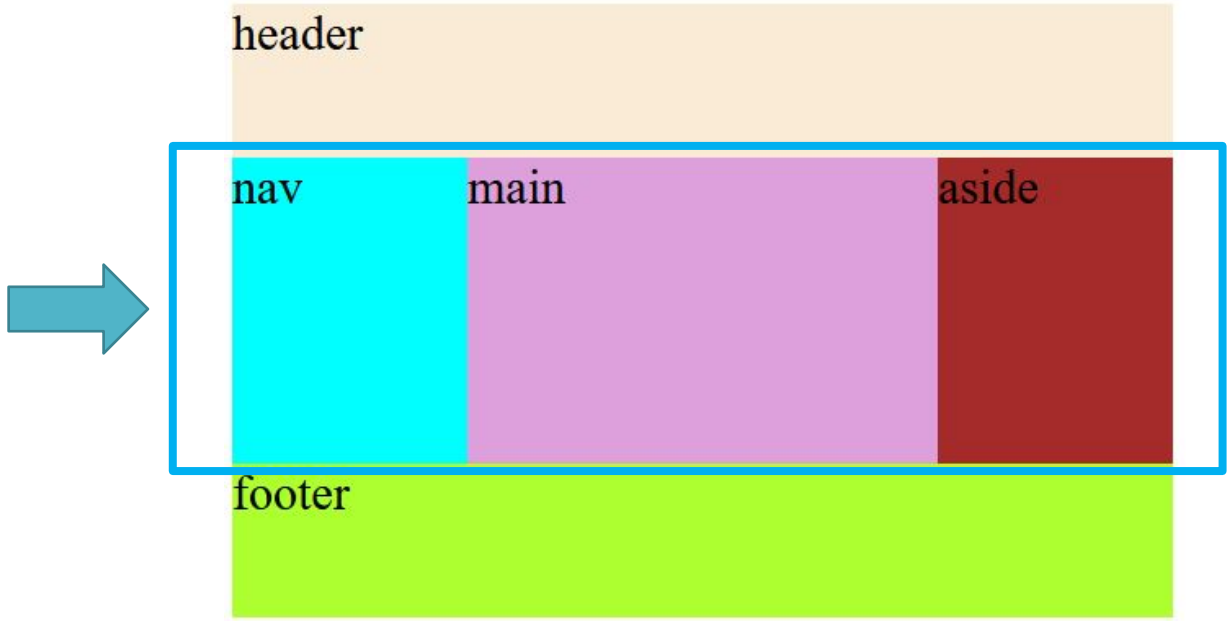
align-self property specifies the alignment for the selected item inside the flexible container.

Its overrides the default alignment set by the container's align-items property




```
<div class="flex-container">
  <div>1</div>
  <div style="align-self: flex-start">2</div>
  <div style="align-self: ">3</div>
  <div>4</div>
  <div style="align-self: center">5</div>
</div>
```

example




```
.container {
    display: flex;
}
```

```
<header>header</header>

<div class="container">
    <nav>nav</nav>
    <main>main</main>
    <aside>aside</aside>
</div>

<footer>footer</footer>
```

example

- home
- gallery
- about



home gallery about

```
<ul>
    <li>home</li>
    <li>gallery</li>
    <li>about</li>
</ul>
```

```
ul {
    display: flex;
}
```

UL -> Flex container

LI -> Flex items

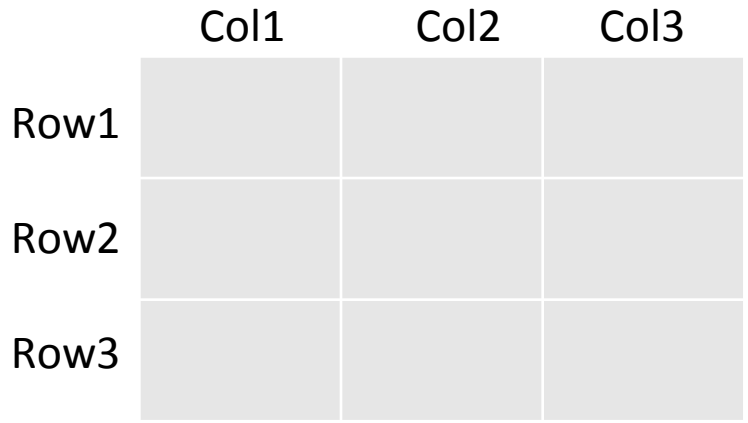
```
li {
    background: skyblue;
    border: 1px solid black;
    padding: 10px;
    margin: 5px;
    list-style: none;
}
```



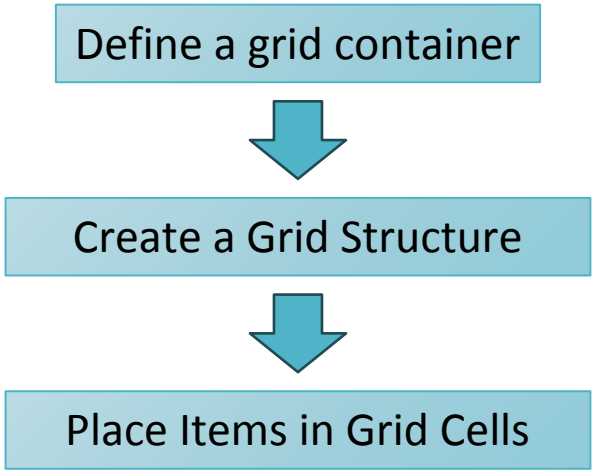
GRID

Grid

Grid is a two-dimensional layout system to create complex and responsive layouts



grid layout having three rows and columns



display: grid

grid-template-rows

grid-template-column

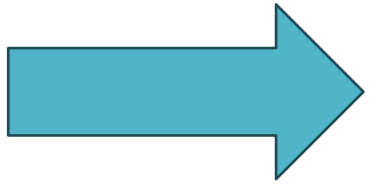
Grid container

grid container is a HTML element that serves as a parent element of the grid items

display: grid - Turns an HTML element into a grid container

```
<div class="container">
    <div>1</div>
    <div>2</div>
    <div>3</div>
    <div>4</div>
    <div>5</div>
    <div>6</div>
    <div>7</div>
    <div>8</div>
    <div>9</div>
    <div>10</div>
    <div>11</div>
```

```
.container{
    background-color: blanchedalmond;
    display: grid;
}
```



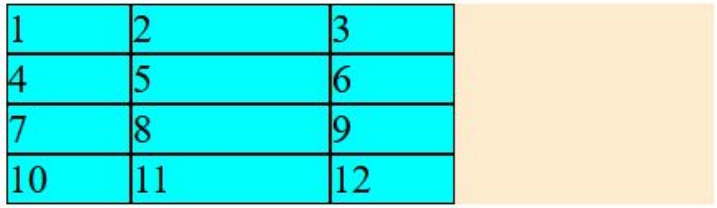


Grid container – grid-template

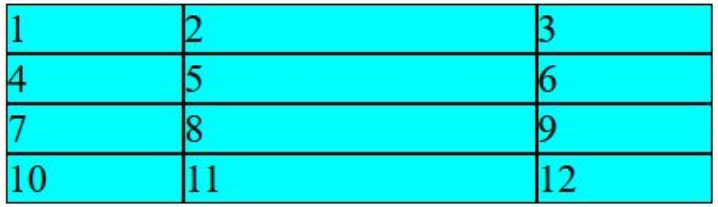
GRID-TEMPLATE-COLUMNS

defines the size and the number of columns within the grid container

```
grid-template-columns: 50px 80px 50px;
```



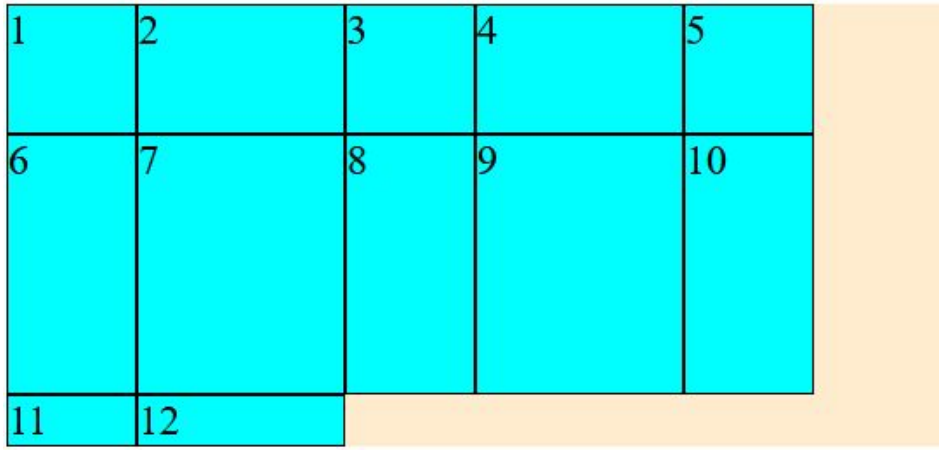

```
grid-template-columns: 25% 50% 25%;
```



GRID-TEMPLATE-ROWS

defines the size and number of rows in a grid container

```
grid-template-columns: 50px 80px 50px 80px 50px;
grid-template-rows: 50px 100px;
```



fractional unit (fr)

fractional unit (fr) divides the available space in the grid container into fractions

```
grid-template-columns: 1fr 2fr 1fr;
```

1	2	3
4	5	6
7	8	9
10	11	12

```
grid-template-columns: 1fr 2fr 1fr;
grid-template-rows: 1fr 1fr 2fr;
```

1	2	3
4	5	6
7	8	9
10	11	12

Minimum and Maximum Grid Track Sizes

minmax() function defines the minimum and maximum size for the grid track

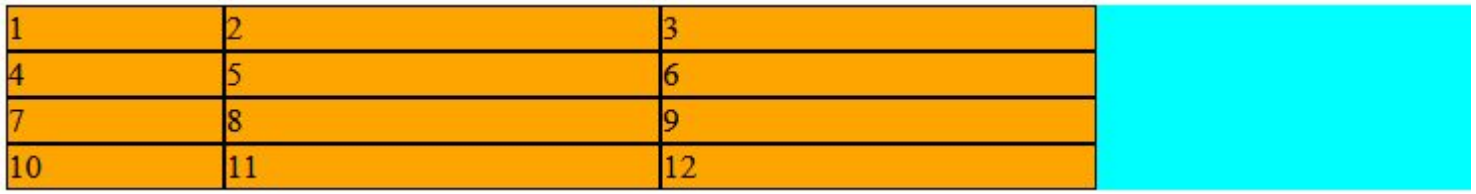
```
minmax(min-size,
max-size)
```

grid-template-columns: minmax(50px,100px) 200px 200px;

min

1	2	3
4	5	6
7	8	9
10	11	12

max

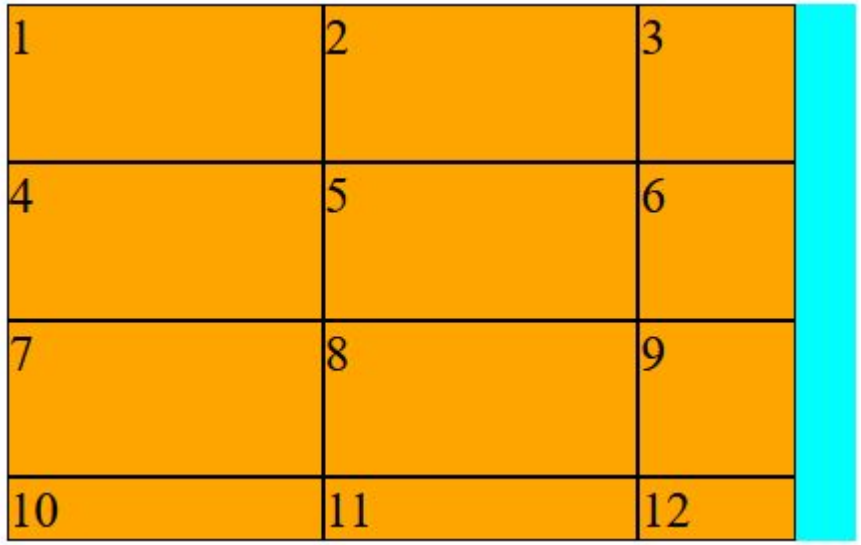


Repeating Grid Tracks

repeat() function defines the repeating grid tracks

repeat(no of tracks, track size)

```
grid-template-columns: repeat(2, 100px) 50px;
grid-template-rows: repeat(3, 50px);
```



Grid container – grid

grid is a shorthand property to specify grid-template-rows and grid-template-columns properties in a single declaration

grid: none | grid-template-rows /
grid-template-columns

```
grid-template-columns: 1fr 2fr 1fr;
grid-template-rows: 1fr 1fr 2fr;
```



```
grid: 1fr 2fr 1fr / 1fr 1fr 2fr;
```

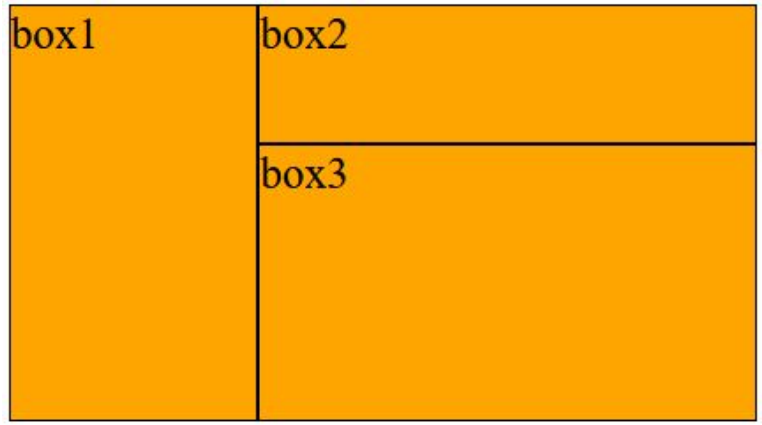
Grid container – grid-template-areas

The grid-template-areas property defines the layout of the grid container using named grid areas

```
grid-template-areas:
  "area-name1 area-name2 area-name3"
  "area-name4 area-name5 area-name6"
  /* ... additional rows ... */;
```

```
.gridContainer {
        background-color: aqua;
        display: grid;
        grid-template-columns: repeat(3, 1fr);
        grid-template-rows: repeat(3, 50px);
        grid-template-areas:
            "box1 box2 box2"
            "box1 box3 box3"
            "box1 box3 box3";
    }
```

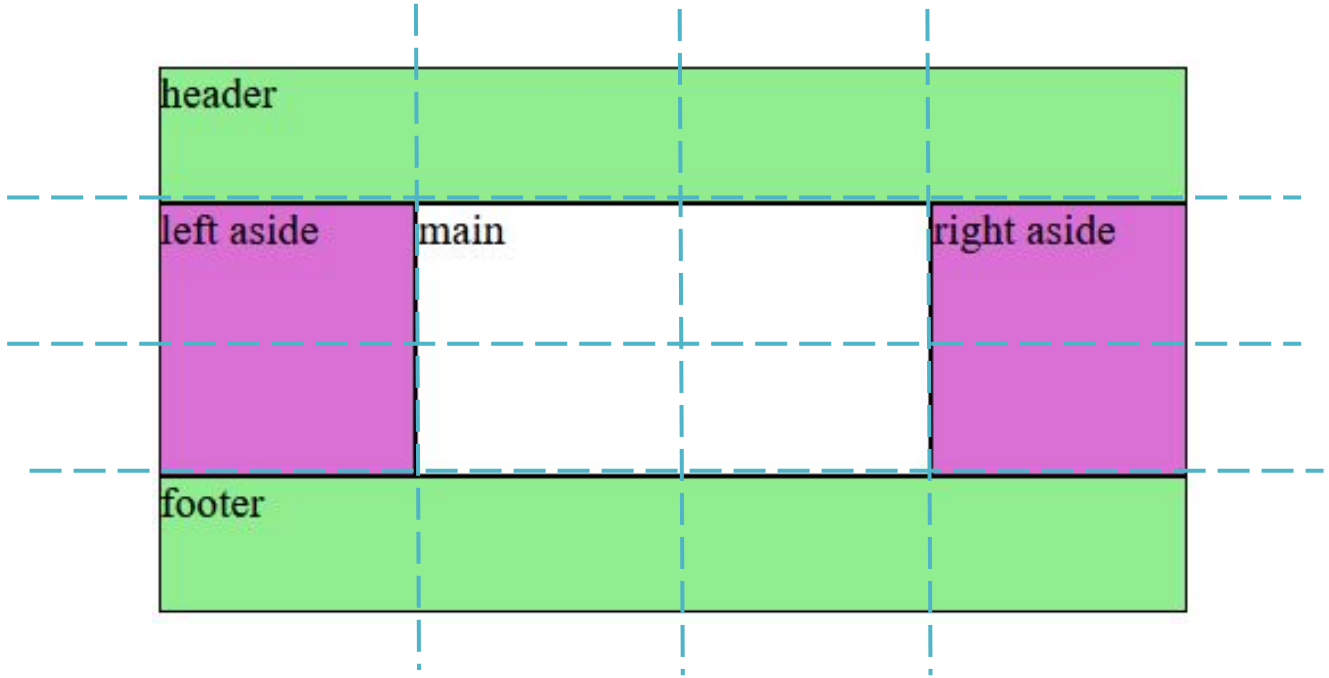
```
.gridContainer>div {
    border: 1px solid black;
    background-color: orange;
}
.box1 {grid-area: box1; }
.box2 {grid-area: box2;}
.box3 {grid-area: box3;}
```




```
<div class="gridContainer">
    <div class="box1">box1</div>
    <div class="box2">box2</div>
    <div class="box3">box3</div>
</div>
```

<style>

</style>




```
<body>
    <div class="container">
        <header>header</header>
        <aside class="left">left aside</aside>
        <aside class="right">right aside</aside>
        <main>main</main>
        <footer>footer</footer>
    </div>
</body>
```

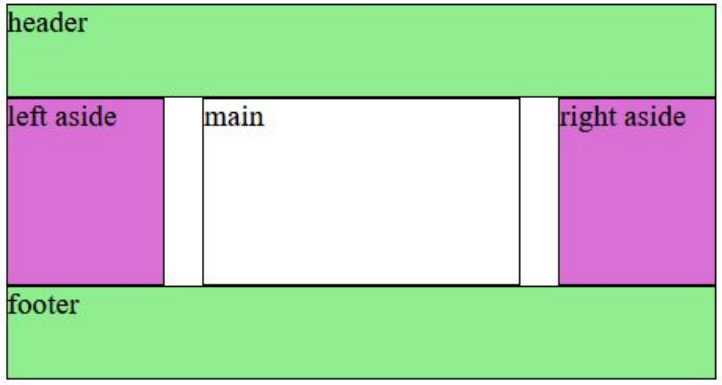
Grid container – grid-gap

grid-column-gap property defines the spacing between columns in a grid container

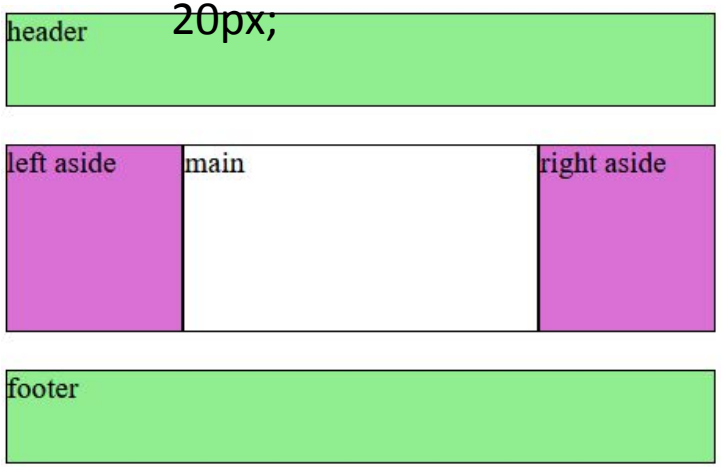
grid-row-gap property defines the spacing between the rows of the grid container

grid-gap is a shorthand property to specify the grid-column-gap and grid-row-gap

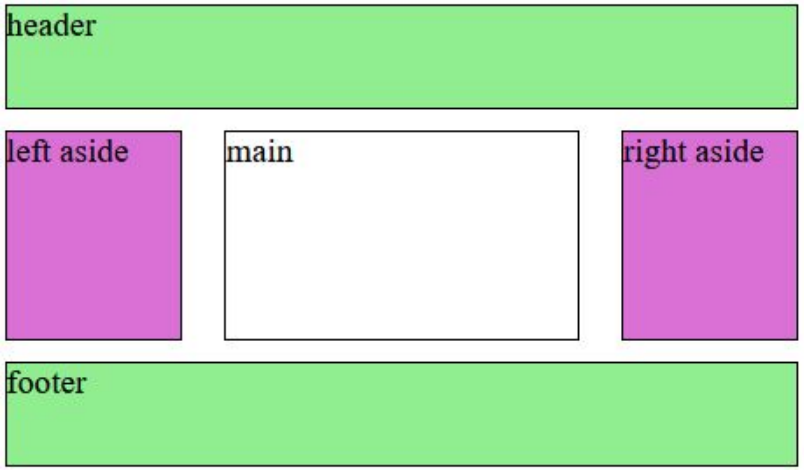
grid-column-gap: 20px;



grid-row-gap: 20px;



grid-gap: 10px 20px;

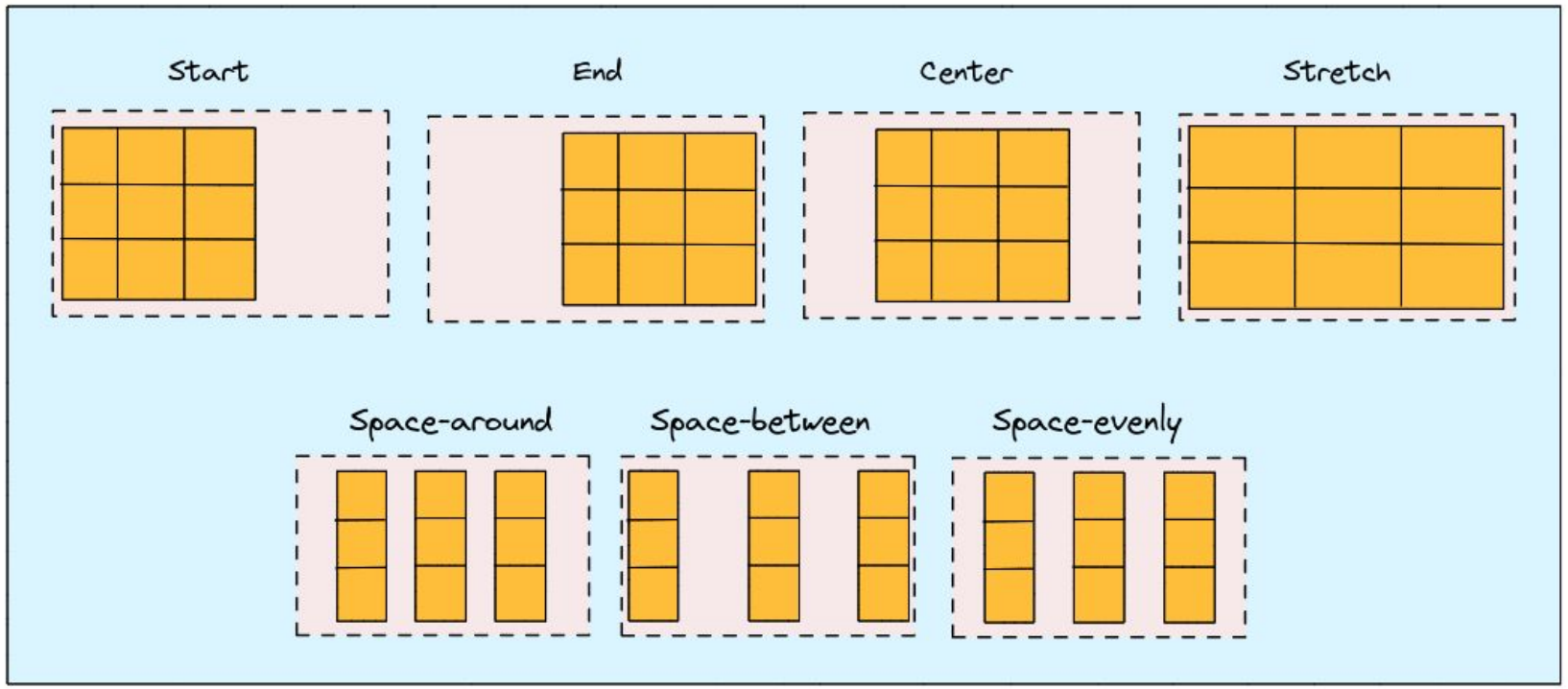


Note grid-gap: 20px; is equal to 20px

Grid container – Justify-content

justify-content property controls the horizontal alignment of the grid within the grid container

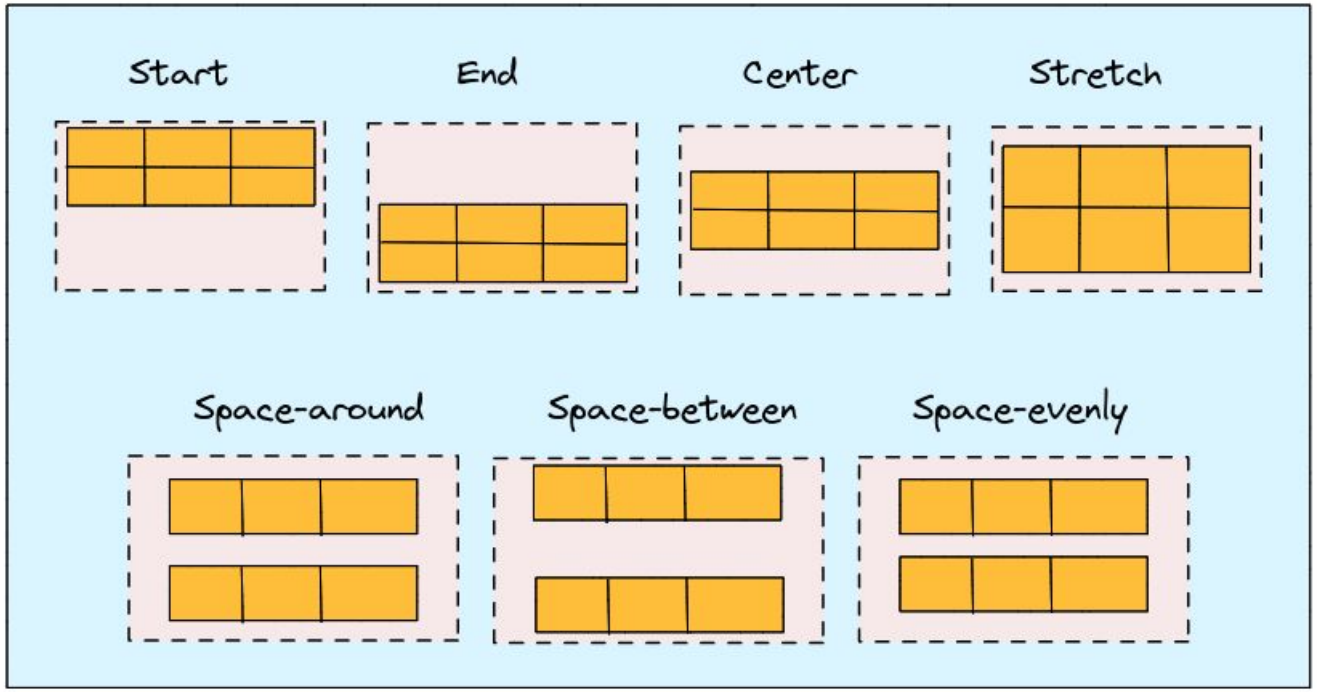
justify-content : start | end | center| space-around| space-between|space-evenly



Grid container – align-content

align-content property controls the alignment of the grid within the grid container along the horizontal direction

align-content : start | end | center| space-around| space-between|space-evenly



Grid container – place-content

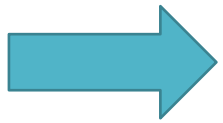
place-content property is a shorthand property to specify the justify-content and align-content in a single declaration

place-content: align-content

place-content

align-content:
center;

justify-content :end

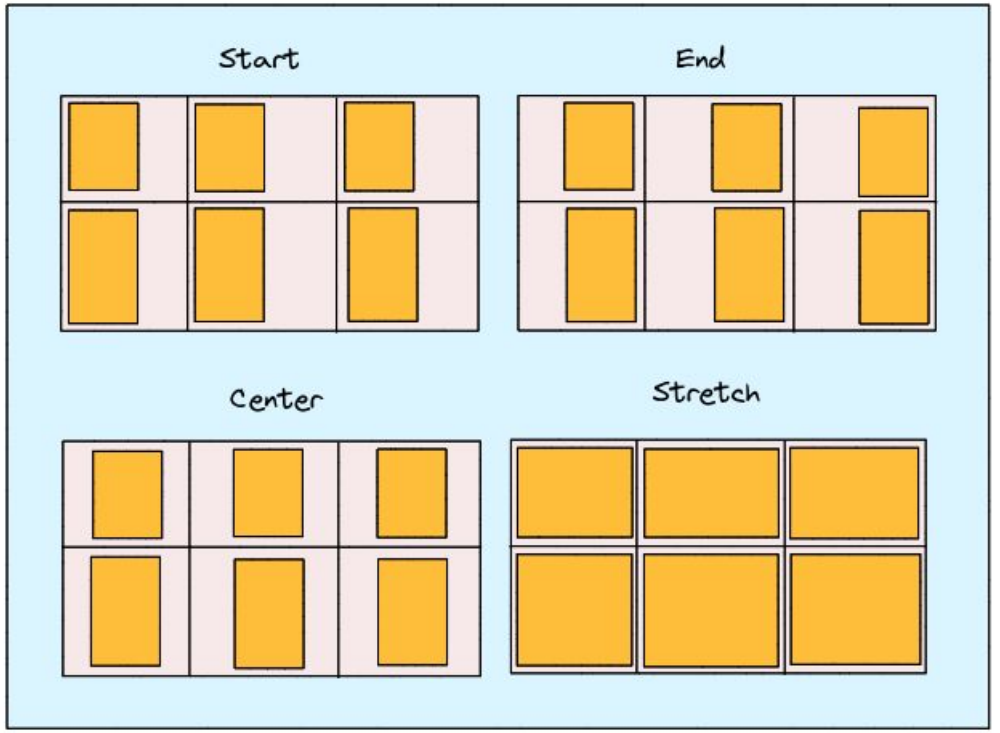


place-content: center
end;

Grid container – justify-items

justify-items property controls the horizontal alignment of the grid items within the respective grid cells

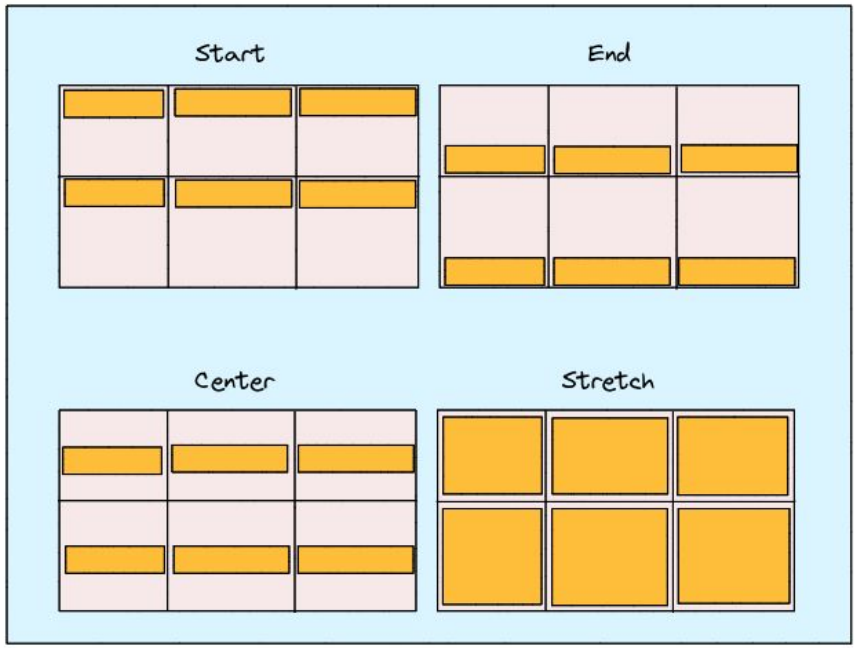
justify-items : start | end | center | stretch



Grid container – align-items

align-items property controls the vertical alignment of the grid items within the respective grid cells

```
align-items : start | end | center |
stretch
```



Grid container – place-items

place-items is a shorthand property to specify the align-items and justify-items property in a single declaration

place-items: align-items

align-items: center;
justify-items:end;



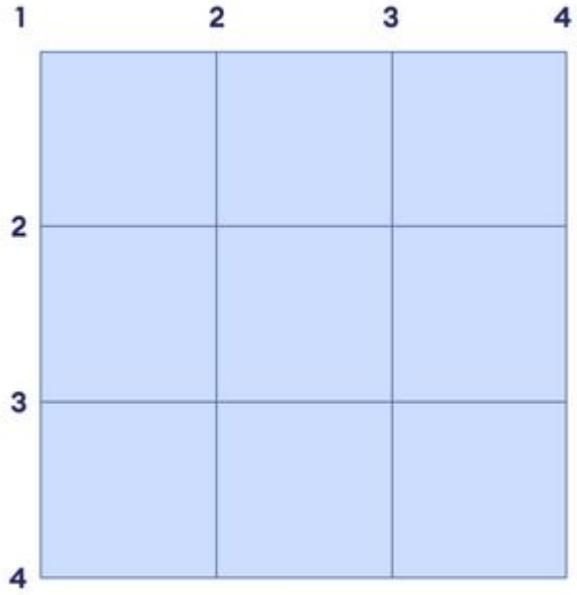
place-items: center end;

Grid items

Grid items are the direct child elements of the flex container



Grid line number



Grid items - grid-row and grid-column

grid-column-start: defines the starting column line number

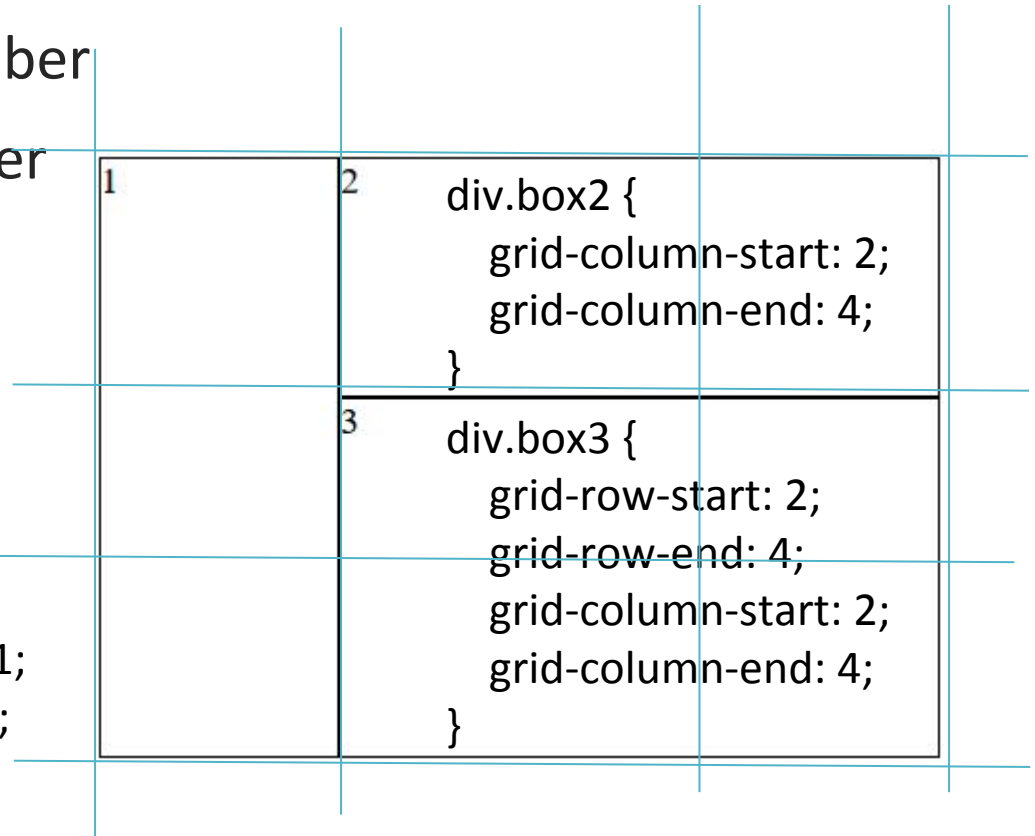
grid-column-end: defines the ending column line number

grid-row-start: defines the starting row line number

grid-row-end: defines the ending row line number

```
.container {
        display: grid;
        grid-template-rows: 100px 100px 50px;
        grid-template-columns: 100px 150px 100px;
    }
```

```
div.box1 {
    grid-row-start: 1;
    grid-row-end: 4;
}
```



Grid items - grid-row and grid-column

The grid-row and grid-column are the shorthand properties to specify the grid line numbers.

```
div.box1 {
    grid-row-start: 1;
    grid-row-end: 4;
}
```



```
div.box1 {
    grid-row: 1/ 4;
}
```

```
div.box2 {
    grid-column-start: 2;
    grid-column-end: 4;
}
```



```
div.box2 {
    grid-column: 2 / 4;
}
```

```
div.box3 {
    grid-row-start: 2;
    grid-row-end: 4;
    grid-column-start: 2;
    grid-column-end: 4;
}
```



```
div.box3 {
    grid-row: 2/ 4;
    grid-column: 2/ 4;
}
```

span

span keyword is used to extend rows and columns by a specified number

```
div.box3 {
   grid-row: 2/ 4;
   grid-column: 2/ 4;
}
```



```
div.box3 {
   grid-row: 2/ span 2;
   grid-column: 2/ span2;
}
```

Grid items - grid-area

grid-area property specifies both the row and column positions of a grid item in a single declaration

grid-area: row-start / column-start / row-end / column-end | item-name;

```
div.box3 {
   grid-row: 2/ 4;
   grid-column: 2/ 4;
}
```



```
div.box3 {
   grid-area: 2 / 4 / 2 / 4;
}
```

```
div.box3 {
   grid-row: 2/ span 2;
   grid-column: 2/ span2;
}
```



```
div.box3 {
   grid-area: 2/ span 2/ 2/ span2;
}
```

Grid items

justify-self property is used to control the horizontal alignment of a grid item within its grid cell

justify-self: start | end | center | stretch | auto

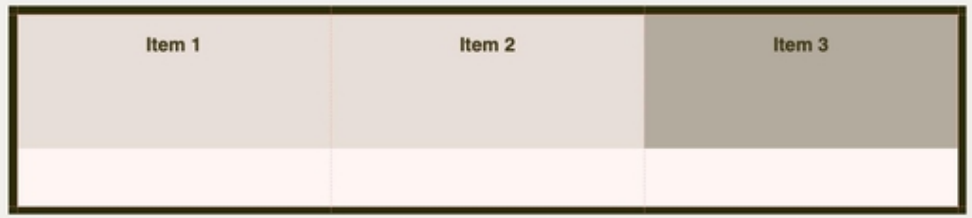
align-self property is used to control the vertical alignment of a grid item within its grid cell

align-self: start | end | center | stretch | auto

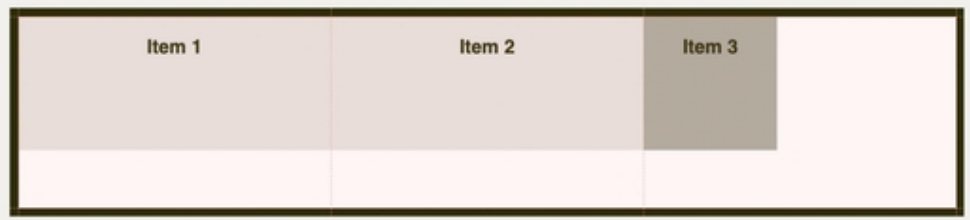
place-self property is a shorthand property to specify the justify-self and align-self property in a single declaration

justify-self

stretch
(normal, auto)



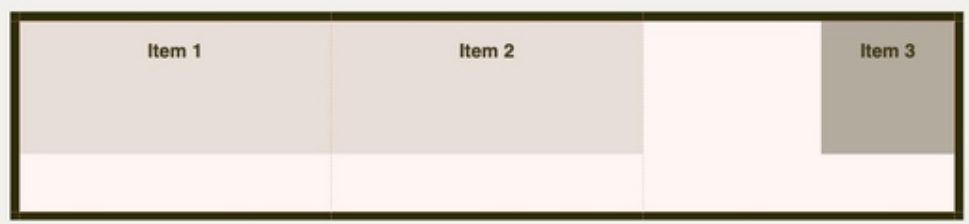
start
(left, self-start)



center



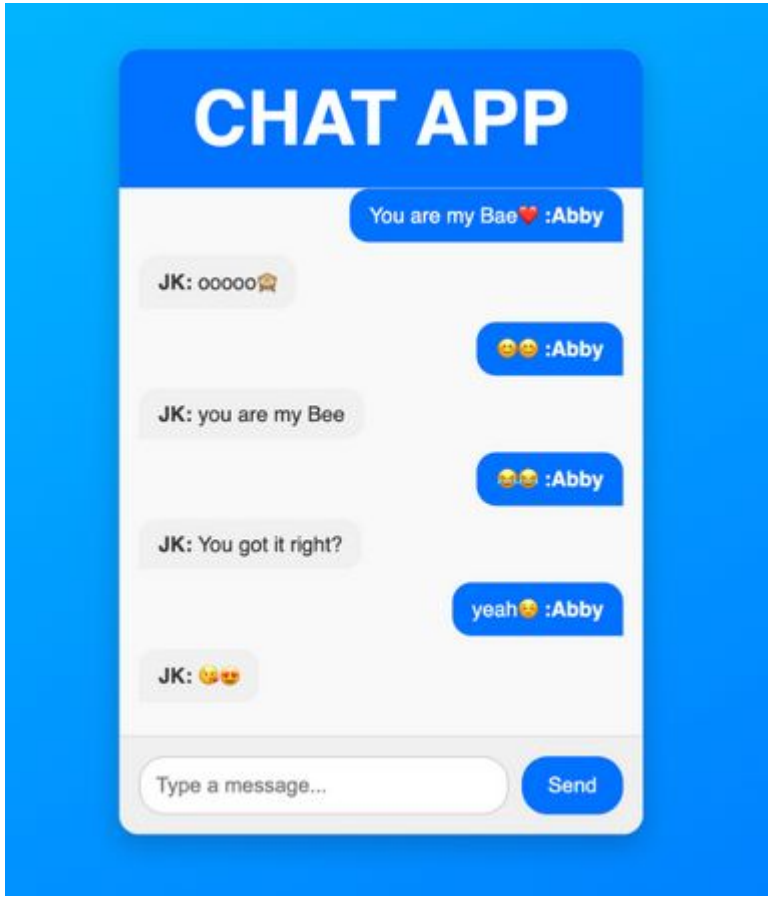
end
(right, self-end)





Todo

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

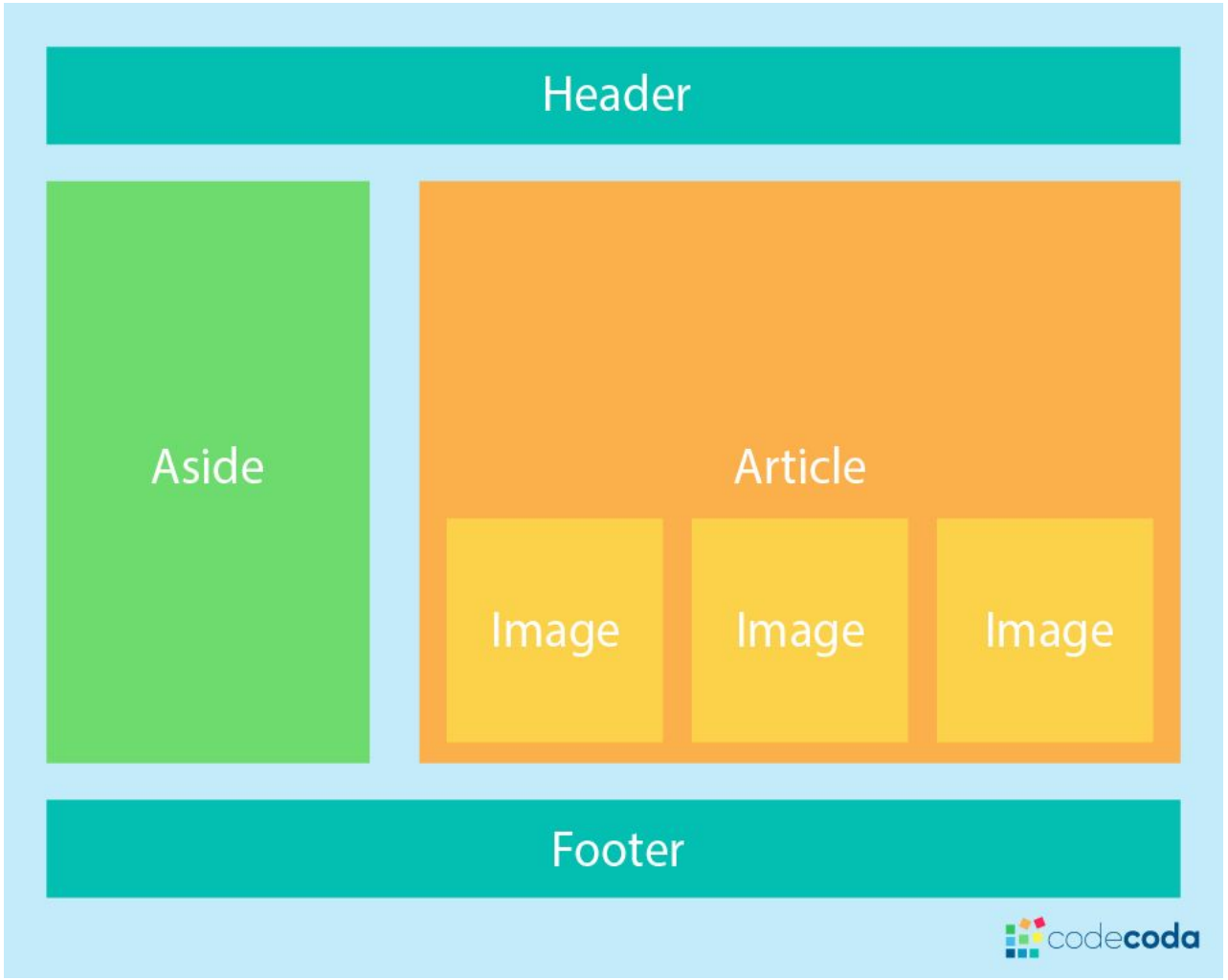



8.3 + 1.7
+
−
×
÷
7
8
9
4
5
6
1
2
3
0
.
C
=

Facebook post



layout



ใช้ semantic tag ให้